Agentic AI-Based Travel Planning System for Indian Tourism

MAJOR PROJECT REPORT submitted in partial fulfillment of the requirements

For the Award of the Degree of

BACHELOR OF TECHNOLOGY

In

INFORMATION TECHNOLOGY

By

DEEKSHITHA SAI KATTA (228W1A1224)

MANOJ KUMAR MANGAM (228W1A1234)

Under the Guidance of

Dr.M.Suneetha, Ph.D,

Professor and Head

DEPARTMENT OF INFORMATION TECHNOLOGY

V R SIDDHARTHA ENGINEERING COLLEGE

(AUTONOMOUS - AFFILIATED TO JNTU-K, KAKINADA)

Approved by AICTE & Accredited by NBA

KANURU, VIJAYAWADA-520007

ACADEMIC YEAR

(2025-26)

V.R. SIDDHARTHA ENGINEERING COLLEGE

(Affiliated to JNTUK: Kakinada, Approved by AICTE, Autonomous)

(An ISO certified and NBA accredited institution)

Kanuru, Vijayawada–520007

CERTIFICATE

This is to certify that this project report titled **"Agentic AI-Based Travel Planning System for Indian Tourism"** is a Bonafide record of work done by **DEEKSHITHA SAI KATTA (228W1A1224) and MANOJ KUMAR MANGAM (228W1A1234)** under my guidance and supervision is submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Information Technology, **V.R. Siddhartha Engineering College** (Autonomous under JNTUK) during the year 2025-26.

(Dr. M. Suneetha)
Dean R&D, Professor& Head
Dept.of Information Technology

(Dr. M. Suneetha)
Dean R&D, Professor & Head
Dept.of Information Technology

EXTERNAL EXAMINER SIGNATURE

Date of examination:

DEPARTMENT OF INFORMATION TECHNOLOGY

V.R. SIDDHARTHA ENGINEERING COLLEGE

PROJECT SUMMARY

S.No	Item	Description
1	Project Title	Agentic AI-Based Travel Planning System for Indian Tourism.
2	Student Names &Numbers	Deekshitha Sai Katta (228W1A1224) Manoj Kumar Mangam (228W1A1234)
3	Name of The Guide	Dr. M. Suneetha, Dean R&D, Professor & Head
4	Research Group	Artificial Intelligence
5	Application Area	Tourism
6	Aim of the Project	The aim of this project is to develop an Agentic AI based travel planning system that can automatically generate travel plans based on user input like budget, location, and number of days.
7	Project Outcomes	1. The system generates accurate and complete travel plans based on user budget, location, and travel duration. 2. It intelligently recommends suitable transport, hotels, and nearby tourist places, helping users plan better and avoid unnecessary expenses.

Student Signatures

1.
2.

Signature of the Guide

ACKNOWLEDGEMENT

First and foremost, I sincerely salute our esteemed institution **V.R SIDDHARTHA ENGINEERING COLLEGE** for giving me this opportunity for fulfilling my project. I am grateful to our Principal **Dr. A.V. RATNA PRASAD**, for his encouragement and support although the way of my project.

On the submission of this Project report, I would like to extend my honor to **Dr. M. Suneetha**, **Ph.D., Dean R&D, Professor & Head of the Department, IT** for her constant motivation and support during the course of my work.

I feel glad to express my deep sense of gratefulness to my project guide **Dr.M.Suneetha, Ph. D., Dean R&D, Professor & Head of the Department, IT** for her guidance and assistance in completing this project successfully.

I would also like to convey my sincere indebtedness to all faculty members, including supporting staff of the Department, friends and family members who bestowed their great effort and guidance at appropriate times without which it would have been very difficult on my part to finish the project work.

TABLE OF CONTENTS

ACKNOWLEDGEMENT	iii
LIST OF FIGURES	v
LIST OF TABLES	vi
ABSTRACT	vii
CHAPTER-1 Introduction	01 -13
1.1 Origin of the Problem	01
1.2 Basic definitions and Background	03
1.3 Problem Statement	07
1.4 Societal Applications of Proposed work	09
1.5 Applications	11
CHAPTER-2 Review of Literature	14 -23
2.1 Description of Existing Systems	14
2.2 Summary of Literature Review	18
2.3 Software Requirements Specification	21
CHAPTER-3 Proposed Methodology	24 -37
3.1 Design Methodology	24
3.2 Dataset Description	25
3.3 System Architecture	28
3.4 Description of Algorithms	31
CHAPTER-4 Results and Observations	38 – 54
4.1 Description of Results	38
4.2 Result Analysis	46
4.3 Observation From work	51
CHAPTER-5: Conclusion and Future Work	55 - 60
5.1 Conclusion	55
5.2 Future Work	57
REFERENCES	viii
APPENDIX	x

LIST OF FIGURES

Figure	Page
Figure 3.1 Architecture Diagram	28
Figure 4.1 Dashboard	38
Figure 4.2 Indetailed Information	39
Figure 4.3 Routing	39
Figure 4.4 Plotted Locations	40
Figure 4.5 Residual Analysis	40
Figure 4.6 Error Distribution	40
Figure 4.7 Graphical View	41
Figure 4.8 Agentic AI travel planner	41
Figure 4.9 Personalized Travel Plan	42
Figure 4.10 ReAct Loop	43
Figure 4.11 Budget Accuracy Comparison	48
Figure 4.12 Plan Completeness Comparison	48
Figure 4.13 Transport mode Correctness Comparison	49

LIST OF TABLES

Table Name	Page
Table 2.1 Literature Review	20
Table 3.1 Prophet Algorithm	32
Table 3.2 Isolation Algorithm	32
Table 3.3 KNN-Harvesine Algorithm	33
Table 3.1 ReAct Algorithm	34
Table 3.2 Harversian Distance Algorithm	35
Table 3.3 Budget Allocation and Reallocation	36
Table 4.1 Test Scenarios and Outcomes	44
Table 4.2 Budget Allocation	44
Table 4.3 React Loop Iteration	45
Table 4.4 Places Agent Performance	46
Table 4.5 Comparison of AI vs Agentic AI	47
Table 4.6 LLM as a Judge Score	50

ABSTRACT

Planning a trip on a limited budget is the problem which is even more complicated by the fact that there are numerous options available, such as transport, hotels and places to visit. Most of the existing systems either have fixed rules or fail to respond accordingly when the conditions or budget vary. This project seeks to address this issue by developing an Agentic AI based travel planning system that can develop a full plan depending on the inputs of the users.

It is a system based on a Large Language Model (GPT-4o-mini) that includes a decision maker and operates with multi-agent system. The agents have functions, calculating distance, cost of travel, budget control, choice of hotels and places to visit. The system does not run in a definite sequence but rather by a ReAct framework, which works on modifying the plan whenever necessary. An important design decision in this system is to separate thinking and calculation. Decision making is made using the language model and all calculations such as cost and distance are made using Python agents. This increases accuracy of the system and prevents calculation errors.

Other features of the system include checking invalid actions, defaulting missing data with fallback options and ensuring that final plan does not exceed budget through Budget optimization. The system was tried using various travel scenarios and it could produce viable and useful travel plans. The project shows that by integrating the use of Large Language Model based reasoning and structured computational agents it is possible to design an intelligent, flexible, and reliable travel planning system that can be used in a real world.

Keywords: *Agentic AI, Multi-agent system, ReAct framework, Budget optimization, Large Language Model (LLM).*

CHAPTER-1

INTRODUCTION

This chapter introduces the foundation and motivation behind the project, provides the basic definitions and background knowledge necessary to understand the work, outlines the problem statement, discusses the societal impact of the proposed work.

1.1 ORGIN OF PROBLEM

Over the past years, the Indian tourism sector has experienced a rapid growth with the growing availability, online awareness and travel destination diversification. Due to the emergence of social media networks, online travel blogs, and online booking systems, there is an increasing number of individuals that plan their trips on leisure, religious or cultural trips. However, with this growth, the process of travel planning has been complex, particularly among domestic travellers who use manual research and numerous sources of information.

Conventionally, tourists would use the Internet to make their travel arrangements by visiting numerous sites to compare options of travelling, to check the availability of accommodation and to visit places of interest via the Internet. This process is usually done by alternating various applications like Google Maps, travel websites, hotel reservation sites and forums. Consequently, users waste a lot of time gathering and processing information before making their travel arrangements. Not only does this disjointed thinking add to the burden of cognitive processes, but it also causes confusion and incoherent decision-making [1].

So, to address this issue, we proposed a system which plans the entire trip for the user's interested location with the Integration of various elements like route selection, transportation, accommodation, budgeting and place recommendations which is one of the biggest challenges in the modern travel planning process. Most of the existing systems consider these components in isolation without looking at the interdependence between the components. As an example, a user can choose a destination based on the interest and then find that the cost of transportation is beyond his or her budget or there is no appropriate accommodation in the desired price range. The lack of a single system that may consider all constraints at the same time leads to such problems.

The deficiency of customization in the conventional travel systems is another major issue. Despite the availability of numerous platforms that give recommendations, they tend to be generic and not user specific. Users differ in their needs depending on their budget, length of travelling, number of people, and interests. The current systems are not adapted to these factors dynamically leading to the provision of recommendations that might not be in accordance with the expectations of the users. This makes the users less satisfied and can create the inefficiency of traveling.

Recent developments in artificial intelligence have provided new opportunities in automating complicated tasks in decision-making. Specifically, the development of large language models (LLMs) and agentic AI systems have demonstrated encouraging outcomes in their capability to deal with multi-step reasoning and executing tasks. Research studies have shown that agentic systems can automatically generate travel itineraries by interpreting user inputs and combining various elements of travel planning. Nonetheless, such systems remain at an initial phase of development and have a few drawbacks [2].

The inability of language models to work with complex constraints is one of the main limitations noticed in the current research. Travel planning task benchmark studies demonstrate that even advanced models have problems with consistency, adequate tool usage, and meeting various constraints like budget, time, and feasibility. These drawbacks underscore the necessity of better system structures that can facilitate systematic reasoning and decision-making.

In a bid to overcome such challenges, researchers have discussed multi-agent systems, in which two or more specialized agents work together to accomplish various tasks. An illustration is that one agent can be charged with the route planning, another can be assigned the cost estimation, and others can be assigned with the accommodation and recommendations. This makes it more efficient and modular, yet lack of coordination and dynamic interaction between agents in many of the existing systems. Consequently, they work in a fixed way and cannot change with the evolving user needs.

Moreover, there is a study that investigated hybrid methods where language models are used together with conventional planning algorithms to achieve constraint satisfaction. Although such systems enhance precision, they bring about further complexity and decrease flexibility,

which are hard to apply in a real-world scenario. Balanced approach, which is flexible, intelligent and reliable is still required [3].

Besides technical difficulties, practical problems of budget management also exist. Numerous tourists, particularly in India, make travel arrangements with fixed financial limits. There is no proper budget allocation or validating systems in place and therefore the plans are either impractical or inefficient in budget. This shows the need to add smart budgetary planning to smart travel systems.

Looking at all these obstacles, it can be concluded that there is a necessity to have a smart and intelligent travel planning system that will be able to escape the shortcomings of the current methodologies. This system must be able to recognize the requirements of the user, do multi-step reasoning, coordinate various components and produce optimized travel plans within the constraints.

Thus, the proposed work is dedicated to creating an Agentic AI-based travel planning system where a ReAct-based reasoning mechanism is incorporated with a multi-agent framework. The system is designed to offer a single travel planning solution to integrate route optimization, transportation choice, accommodation suggestion, budget allocation, and place suggestion into one intelligent system. This not only streamlines the process of planning but also improves decision making in that travel planning becomes easier, more customized and dependable [4].

1.2 BASIC DEFINITIONS AND BACKGROUND

1.2.1 Travel Planning Systems

Travel planning systems are software programs that help users to plan travel by giving recommendations on route, transport, accommodation, and tourist sites. The traditional systems are mostly rule and are based on pre-established datasets or fixed recommendations. Although these systems are helpful in simple planning, they do not have the capabilities of adapting to dynamic situations and user requirements.

Contemporary travel planning application uses AI to give personalized suggestions. The systems work on user inputs like preferences, budget and travel time to produce tailored

itineraries. Nevertheless, there are numerous systems that are already in place, but they are still limited and only serve as individual solutions. As an illustration, a common system might give suggestions of the most popular tourist destinations, but it does not factor in the possibility of the user to afford the trip or whether the recommended route is the best one. On the same note, there are systems that are dedicated to the reservation of services, but do not give ideas on how the trip should be organized.

The suggested system would address these shortcomings by incorporating all the elements of travel planning in one system. It incorporates agentic AI, multi-agent systems and reasoning mechanisms to offer a holistic approach to travel planning, which considers all elements of the travel planning process at the same time [1].

1.2.2 Agentic AI

An agentic AI is a type of artificial intelligence system that can make autonomous decisions, meaning that such a system can plan, act, observe and update its decisions independently without the need to be continuously monitored by a human. In contrast to the conventional AI systems that can be described as reactive i.e. merely as reacting to the user input, agentic AI systems can be viewed as intelligent entities that pursue goals. The process in an agentic system normally takes a reasoning and action cycle. The system initially takes the request of the user and develops a plan. It then executes a series of operations, including accessing data sources or external tools, or communicating with other components. Depending on the outputs of such actions, the system monitors the results and changes its internal state and moves to the next step. This recursive method enables the system to deal with highly complicated, many-step processes.

But agentic AI is significant in that it can manage issues of the real world, where humans need to make dynamic decisions. As an example, a user can input things like number of days, budget and destination in the case of travel planning. The system then must decide on the optimal path, the transport to use, the accommodation to use, and places of interest to visit- all within the constraints. Such complexity would be a challenge to a conventional system, but it can be approached systematically by an agentic AI system which can further subdivide the problem into smaller steps.

Within the framework of the proposed project, agentic AI will serve as the primary layer of intelligence that implements the entire planning process. It helps the system to coordinate various modules, make decisions based on intermediate results, and optimize the travel plan until it satisfies all the requirements. This shapes the system to be more flexible, adaptive and able to deal with various user situations [2].

1.2.3 Multi-Agent Systems

A multi-agent system (MAS) is a design where several intelligent agents are collaborating to solve a problem. Agents are supposed to do a given task and when combined, they help the accomplishment of a shared objective. This method is especially applicable to complicated issues that may be broken down into smaller, independent tasks.

Various agents may be allocated specific roles in a multi-agent travel planning system. As an illustration, one agent can perform the task of finding routes and distances, the other one can choose transportation methods, another agent can find the budget and the other agent can suggest tourist destinations. The system is also more organized and efficient because it allocates responsibilities to the agents.

Modularity is one of the major benefits of multi-agent systems. Agents are independent and, as a result, it is easier to update or enhance the individual components without compromising the whole system. Also, the agents will be able to interact with one another, exchange information and organize their actions to generate a consistent output.

But the coordination of agents is a very important challenge. In the absence of orchestration, agents can yield conflicting or inconsistent results. Thus, there must be a central controller, or more commonly known as an orchestrator, to coordinate agents with each other, and to guarantee that all the tasks are done in a rational order.

The multi-agent architecture of the proposed system is very important in subdivision of the travel planning process into manageable parts. The agents are specialized in one part of the problem, and the orchestrator makes sure that every agent collaborates in producing a full and optimal travel schedule [2].

1.2.4 Budget Allocation and Optimization.

Budget allocation is a major issue to consider when planning a trip, particularly to those users with limited financial means. It also entails setting up the overall budget which is available to be divided into various categories like transportation, accommodation, food, and activities. The adequate allocation also allows the travel plan to be viable and does not result in excessive expenditures.

Traditional systems leave budget management to the user, and this can result in inefficient planning. Users could spend excessive budget on a single component and have inadequate money to spend on other components. This may lead to half-baked or unrealistic travel plans.

The budget allocation should be done automatically through an intelligent system that will examine the inputs of the user and allocate the budget among various components in the most optimal way. It also needs to confirm the final plan so that the overall cost would not be more than the budget available.

This is a task that a special budget agent would oversee in the proposed system. It is dynamic in terms of its budgetary allocation in accordance with the options chosen and makes sure that the end plan is realistic and affordable. This greatly enhances user experience and removes the possibility of manually computing budgets [3].

1.2.5 Calculation of Location and Distance

Accurate location and distance calculation are essential for effective travel planning. The calculations assist in the determination of the viability of routes, calculation of the travel time, as well as the choice of the suitable transportation methods. The Haversine formula is one of the commonly used formulas to calculate the distance between two geographic locations. This formula is used to calculate the shortest line between two points on the earth surface basing on the latitude and longitude of the points. It has extensive applications in navigation and mapping.

Distance calculation is important in making choices in travel planning. To illustrate, shorter routes might be favoured by road modes of transport whereas long routes might need air

transport. There is also the distance information which can be utilized in estimating the costs and time taken in travelling. The route agent calculates the distance in the suggested system to find the most suitable paths and suggest appropriate transportation modes. This makes the travelling plan efficient and feasible [3].

1.2.6 ReAct Framework

ReAct framework is a recent style of artificial intelligence that integrates the reasoning and action into one system. The conventional AI models tend to ignore the two processes, which causes them to be limited when it comes to complex jobs. ReAct addresses this issue by enabling the system to perform reasoning and action in an interleaved manner.

The framework works in a loop of three primary actions in thinking, acting, and observing. The system produces reasoning steps which lead to its decision making during the thinking phase. In the acting phase, it performs actions such as calling an external tool or querying a database. This is done by evaluating the outcome of its activities in the observing stage and updating its knowledge to the next stage.

By doing this, the system can dynamically refine its decisions. As an example, when an option of transport that has been selected is beyond the financial capacity of the user, the system can identify the problem and select an alternative during the next round. This flexibility and capability to rectify errors is one of the strengths of the ReAct framework. The other significant advantage of ReAct is enhanced transparency. This is because the system explicitly produces reasoning steps, thus users can know how the decisions are made. This enhances credibility and the system becomes more interpretable. The orchestrator which is used in the proposed project to have all the agents coordinated is implemented using the ReAct framework. It allows the system to reason in multi-steps, communicate with various agents, and optimize the travel plan in an iterative manner until it meets all the constraints [4].

1.3 PROBLEM STATEMENT

Travel planning is a multi-dimensional process that incorporates the choice of destinations and routes, mode of transport, accommodation, budget management, and places of interest. In the real-world contexts present today, many travellers use a variety of disconnected systems like

travel sites, reservation application, maps and reviews to obtain the required information. This disjointed nature renders the planning process time-consuming, inefficient, and in most cases confusing particularly to the less experienced user [3].

Lack of intelligent coordination among various elements of the planning process is one of the biggest challenges of the current travel planning systems. Most systems work in isolation, paying attention to a particular feature like hotel booking, transport choice, or place suggestions. Due to this, users must hand-assemble these aspects, which in most cases presents inconsistencies and non-optimal decisions. To explain this point, a user can select a destination and a hotel without knowing that the transportation cost is more than the available budget, or the places which cannot be afforded within the specified period.

The other important problem is the inefficiency of the current systems to address numerous constraints at a time. The process of travel planning is associated with a few constraints: budget, time, distance between two places, and preferences of users. Conventional systems either do not acknowledge these constraints or address them in a constrained way. Even AI systems that are currently in use, especially large language models, find it difficult to adhere to consistency and meet all constraints in complicated situations. Research has revealed that such models tend to produce logically inconsistent plans, or plans that do not satisfy the needs of the user because of absence of structured reasoning and iterative decision making.

Moreover, most travel planning systems do not have dynamic adaptability. Conditions like cost, availability and user preferences of travels may vary quickly and therefore the system needs to modify its decisions. But current systems are usually static in their output, which is not updated according to new information. This restricts their usefulness in practical situations where flexibility and adaptability are necessary. Multi-agent systems are proposed as a possible solution to enhance modularity and efficiency through task allocation among the specialized agents. As much as this is an excellent way of improving performance, a lot of the current applications are poorly coordinated and orchestrated. In the absence of a central reasoning mechanism, the agents can generate disjointed outputs and therefore incomplete or suboptimal travel plans.

Also, budget management is another major issue, especially to the users in some areas such as India where cost is a major factor in the decision to travel. Most of the systems lack adequate

budget allocation or validation systems resulting into plans that are more than the budget or plans that do not make efficient use of the budget. This underscores the importance of having a smart system that is capable of dynamically controlling and maximizing budget allocation to various aspects of the travel planning. Lack of transparency in decision-making is another limitation. Most AI-based systems operate as black boxes, whereby they give results without justifying their basis. This decreases user confidence and complicates the process of checking the accuracy of the plans produced. It is required to have systems that deliver results and clarify the decision-making process.

Based on all these issues, it is evident that there is a necessity to have a standardized and smart travel planning system, which can incorporate various elements, manage constraints, and dynamically adjust to changing circumstances. The system must be able to reason in a multi-step manner, coordinate several specialized agents and produce optimized travel plans that meet the needs of the user.

1.3.1 Objectives

- Develop an AI-based intelligent travel planning system
- Optimize travel routes based on distance and feasibility
- Ensure trip planning within user-defined budget constraints
- Provide a simple and interactive web interface

1.3.2 Outcomes

- Automatic generation of complete day-wise travel plans
- Reduce trip planning time from hours to seconds
- Achieve budget-aware travel planning
- Improve user convenience and decision-making

1.4 SOCIETAL IMPACT OF PROPOSED WORK

- **Enhances efficiency of travel and lessens user effort**

Among the key effects of the suggested system, it must be noted that it will simplify the process of travel planning. In the modern context, users take a significant amount of time to find the destinations, compare the means of transportation, see the availability of hotels, and

estimate the prices. This can entail the use of various platforms, which can cause confusion and inefficiency. The proposed system automates all this by coming up with a complete travel plan based on user inputs like budget, destination and duration. It will greatly decrease the amount of effort needed in the planning process by uniting several elements into one system. Within a few seconds users can receive a well-organized itinerary, which does not require any manual research and choice. Not only does this save time but it also lessens cognitive load, and thus travel planning is easier and more accessible, particularly to users who do not use digital tools or travel platforms [1].

- **Improves decision making and minimizes mistakes**

The travel planning process is associated with several decisions, which tend to be interdependent. Misjudgement like choosing a costly mode of transport or wrong hotel can result in wasting money or being inconvenienced. The orthodox systems lack adequate support in generating the best decisions, and users are left to use the trial-and-error solutions. The suggested system enhances decision-making by applying agentic AI and coordination among the agents to evaluate the alternatives in a systematic manner. It considers various variables including cost, distance, and preferences of the user and then comes up with recommendations. Also, decision-making based on reasoning mechanisms makes the decision logically consistent and aligned to constraints. The system helps to reduce human mistakes and offer the most efficient solutions, which improves the quality of travel planning overall and makes users more confident [2].

- **Economic growth and encourages local tourism**

Tourism plays a significant role in economic development especially in those areas that have well-developed cultural and natural features. Nonetheless, most of the time the benefits of tourism are concentrated on few of the popular destinations, leaving the less popular destinations underutilized. The suggested system will be able to facilitate equal tourism by suggesting different sites according to the preference of the user and the location information. It is promoting a more diverse selection of destinations and less well-known places by proposing these options to tourists [3]. This translates into more economic opportunities to the local communities such as small businesses, hotels, restaurants and transport providers. It also promotes rural tourism and evenly distributes tourism income in various regions.

- **Enhances access by any type of user**

The second significant societal effect of the system is that it has enabled the travel planning to be accessible to more people. Several of the current traveling websites need one to possess some technical skills or expertise in operating more than one tool. This poses a challenge to non-digitally savvy users. The proposed system also makes the interaction process easier since it offers an easy-to-use interface and automatic planning. All that the users do is to make simple input, and the rest is done by the system. This renders travel planning available to people with various backgrounds such as students, families and the elderly. The system will make intelligent travel planning solutions accessible to a greater number of people due to its simplified nature and increased usability [4].

- **Favors sustainable and efficient traveling planning**

The modern travel planning is a significant element of sustainability. This may result in excessive fuel use, traffic jam, and environmental degradation due to inefficient travel routes, unnecessary routes, and poor planning. The suggested system is sustainable in the sense that it minimizes travel routes and suggests effective means of transport. It saves on unnecessary travel and encourages the use of resources efficiently by considering the distance and cost factors. Moreover, the system aids in the reduction of congestion along with tourist destinations by spreading the traffic to various places, which in turn reduces environmental pressure and conservation of natural and cultural sites.

1.5 APPLICATIONS

- **Intelligent travel planning for tourists and individuals**

The system has one of its major uses in guiding individual travellers in planning their trips effectively. Tourists are usually challenged in destination choices, budgets and itinerary planning. The proposed system makes this process easy by automatically creating a full-fledged travel plan based on the user inputs in terms of destination, budget and duration. The personalized recommendations of routes, transportation, accommodation, and places to visit are available to users without using several platforms to search the information manually. This saves time as well as enhances the overall travelling experience by offering enhanced and perfectly designed itineraries [1].

- **Inclusion in travel and tourism portals.**

The system can be incorporated in the current travel websites, mobile applications and booking systems to facilitate their functionality. The existing travel solutions are mainly associated with the reservations of services like flights and hotels but do not possess smart planning features. With the implementation of the proposed system, these platforms will be able to provide such advanced functionality as automated itinerary creation, budget optimization, and customized recommendations. This would give it a competitive edge and enhance customer satisfaction.

- **Decision-support system for travel agencies**

The system can be employed by the travel agencies as a decision-support tool to better serve their customers. Travel agents can use the system to develop optimized itineraries in a short time using it instead of developing travel plans manually. This will decrease the workload and enhance efficiency as well as enable agencies to handle more customers within a shorter period. Moreover, the system will also guarantee accuracy and uniformity in travel plans and eliminate the possibility of errors [6].

- **Research and educational uses**

As a research tool, the system can be applied in the study of intelligent systems, multi-agent architectures and decision-making models. Students and researchers will be able to study the application of agentic AI to real-world problems and test various algorithms and architectures. It is also a practical representation of the way concepts like reasoning frames, agent coordination and constraint management can be applied in the real world.

- **Creation of AI-driven personal assistants**

The suggested system may be used to create powerful AI-based personal assistants that can be used to perform more complicated tasks beyond what travel planning does. Through the extension of the architecture, the system can be modified to support the users in other fields like event planning, day-to-day scheduling, and management of resources. This demonstrates the versatility of agentic AI systems and their potential to transform how users interact with technology [7].

- **Business and enterprise software.**

The system can be used to optimize the services offered by organizations in the tourism, hospitality and transportation sector. As an example, hotels can use recommendations to change pricing strategies, and the transport providers may plan the routes and schedules according to demand. The system can also be used to improve customer engagement by providing personalized recommendations and enhancing user experience**.**

- **Integration with smart and real time systems in future**

It is possible to add more real-time data sources, including weather, traffic conditions, and real-time availability of services to the system. This would allow the process of dynamic travel planning where the system would constantly update the itinerary in response to changing conditions. This kind of integration would make the system more robust and able to be used in the real world in smart cities and digital ecosystems [8].

This chapter described the different uses of the system in areas like tourism, business, research, and intelligent digital systems. These applications show the flexibility and extendibility of the suggested solution that can be incorporated into travel applications and smart assistants.

CHAPTER – 2
REVIEW OF LITERATURE

This chapter describes the review of literature that we have taken from various papers and considered all the points mentioned in the papers.

2.1 DESCRIPTION OF EXISTING SYSTEM

Paper-1 Design and evaluation of an agentic AI framework for personalized Umrah trip planning

The article introduces a detailed agentic model of AI that focuses on the resolution of the multifaceted issue of end-to-end travel planning in terms of Umrah pilgrimage. The paper emphasizes that the conventional travel solutions are disjointed wherein users are left to independently deal with transportation, accommodation, scheduling, and documentation. The net effect of this non-integration is inefficiency, bad user experience, and cognitive load, particularly among first time travelers.

One of the most important strengths of the system is the fact that it uses the structured memory which enables the agent to store the intermediate planning decisions and revert to them to make the subsequent steps of the reasoning process. This allows the system to be consistent at various levels of the planning process and prevent redundant calculations. It can also be integrated with APIs and external utilities via its architecture, which enables it to receive real-time information about transportation, accommodation availability, and scheduling constraints.

The reasoning system used in the system facilitates multi-step planning whereby the agent analyzes a few options before choosing the best solution. This is an iterative reasoning process that enhances the quality of the created itineraries by making sure that all the constraints are met and that the itineraries maximize user satisfaction. Its multilingual itinerary generation is also supported by the system, proving its ability to serve a wide range of users.

The proposed system is directly based on this work, since both systems use agentic AI concepts to plan the automated travel. Nevertheless, the proposed system builds upon this strategy by

introducing a ReAct-based orchestrator, multi-agent specialization, and rigid budget validation to fit an Indian tourism setting [1].

Paper-2 Automated travel planning via multi-agent systems and real-time Intelligence

The article introduces a multi-agent system known as Travel Optix, which is aimed at eliminating the inefficiencies of the traditional travel planning systems. The authors find that users normally use a variety of sources to obtain information about destinations, accommodation, transportation, and cost and therefore it is time-consuming and inconsistent. Also, many of the existing systems offer predetermined recommendations that are not personalized and cannot be modified to the current circumstances at the time of traveling.

To address these issues, the given system suggests a modular multi-agent design in which the agents are given a particular task. These agents cooperate to carry out various functions like destination analysis, itinerary generation, cost estimation, and activity recommendations. The roles are separated so that the agents specialize in a specific area which enhances efficiency as well as precision of the entire system.

Another key characteristic of the Travel Optix platform is that it can integrate with real-time data sources via APIs like Serper and browser less applications. This enables the system to retrieve current information about weather conditions, local events, accommodation as well as prices. The system can be used to create relevant and accurate travel plans by using real-time intelligence, unlike traditional systems that use outdated or static data to generate their travel plans.

Though the system has its benefits, there are some limitations. The dependency on third-party APIs presents potential problems of latency, data access, and scalability. In addition, the system can be problematic where there is a high number of users simultaneously requesting their services and this might interfere with performance.

The main value of this work is that it shows the effectiveness of multi-agent systems when automating complex tasks of travel planning. The system is very efficient and personalized because it allocates responsibilities to specialized agents and incorporates real-time information.

The work is highly related to the suggested system, as it applies a multi-agent framework for planning travel as well. The suggested system, however, builds upon the idea by adding an element of ReAct-based reasoning loop, allowing dynamic replanning and better decision-making than the comparatively fixed workflow in Travel Optix [2].

Paper-3 Travel Planner: A benchmark for real-world planning with language agents

The article presents a novel benchmark named Travel Planner that is aimed at assessing the performance of language agents in a real-world travel planning context. The authors emphasize that planning is one of the key features of human intelligence, where one should be able to accumulate information, analyze constraints, and make sequential decisions. Nevertheless, conventional AI systems have not been able to mimic this ability in complex unconstrained environments.

To fill this gap, Travel Planner benchmark offers a realistic and challenging test environment in which one can test language agents. It consists of a huge amount of travel scenarios with various constraints like budgetary limits, accommodation choices, mode of transportation, and time constraints. The benchmark also provides access to various tools, such as flight search, accommodation search, restaurant recommendations, and distance calculation, which allows agents to engage with external environments.

The paper highlights that iterative reasoning and memory management are significant in the process of solving a real-world planning problem. Travel planning, unlike simple tasks which can be resolved in a one-step process, needs to be evaluated and refined in the decisions made. The fact that current agents cannot engage in such iterative reasoning points to an important shortcoming of the current AI systems.

The paper also makes another valuable contribution regarding constraint handling. There are several constraints that must be met at the same time during travel planning like budget constraints, time constraints and user preferences. The benchmark shows that the existing agents do not usually balance these constraints effectively and as a result, they end up with suboptimal or invalid plans.

This article underscores how current language agents are inadequate to address the intricate travel planning processes. The proposed challenges system tries to solve by using a structured

multi-agent architecture with a ReAct-based reasoning-loop, which allows improved constraint management, tool use, and decision-making in the form of iteration [3].

Paper-4 ReAct: Synergizing reasoning and acting in language models

The article presents the ReAct (Reasoning + Acting) framework that is an important step toward the creation of intelligent systems based on large language models. The authors find that the conventional methods of AI problem-solving consider reasoning and action as distinct processes, which results in inefficiencies and poor performance in complex tasks.

ReAct overcomes this weakness by combining the elements of reasoning and action into a single model, with the model generating reasoning steps (thoughts) and executing actions in an interleaved fashion. This will enable the model to dynamically engage with external tools, collect accurate information and revise its decisions in response to new observations.

The framework is a cyclic process that comprises three key steps, which include reasoning, action, and observation. The reasoning stage entails the model coming up with intermediate thoughts on which it bases its decision-making process. The model in the action stage will react with the external tools or settings to seek more information. Lastly, during the observation stage, the model can update its knowledge depending on the outcomes of its actions and thereby refine its plan.

The results of the experiment prove that ReAct is much more effective in a range of tasks than the traditional ones, such as question answering and interactive decision-making. The framework is also quite successful in situations that involve a multi-step rationale and adaptation.

This system is based on this framework. The orchestrator uses the ReAct loop (THINK–ACT–OBSERVE) to orchestrate various agents and, therefore, facilitates dynamic planning, real-time decision-making, and refinement of travel plans [4].

Paper-5 TRIP-PAL: Travel planning with guarantees by combining large language models and automated planners

The paper by de la Rosa et al. introduces TRIP-PAL, a hybrid travel planning system that combines the capabilities of large language models with traditional automated planning

techniques. The authors emphasize that although LLMs can be effective to generate travel recommendations because they possess a lot of knowledge, they tend to generate logically consistent and constraint-satisfying plans.

To resolve this problem, TRIP-PAL takes a hybrid design in which the information extraction and interpretation is performed by LLMs, and the generation of valid and optimized travel itineraries is carried out by automated planners. The LLM takes the user input and derives useful information like points of interest, travel preferences and constraints. This data is then translated into a format which can be handled by automated planning algorithms.

The automated planner makes sure that the created itinerary meets all the requirements such as time, travel feasibility, and preferences of the user. This ensures that the travel plan is accurate and dependable as one of the biggest drawbacks of the entirely LLM-based systems. The main value of this work is that it shows the advantages of implementing the method of combining symbolic planning methods with data-driven models, which is a balanced solution to complex planning problems.

This paper has much in common with the proposed system, in integrating intelligent reasoning, and structured planning. Nonetheless, rather than using formal planners, the suggested system makes use of various specialized agents and a ReAct-based orchestrator, which provides more flexibility and adaptability [5].

2.2 SUMMARY OF LITERATURE REVIEW

Based on the examination of available research studies, one may note that the recent developments in terms of travel planning systems have been more concerned with the incorporation of large language models (LLMs), agentic AI, and multi-agent systems to automate the process of itinerary generation and enhance user experience. The works of agentic AI-based frameworks and multi-agent travel systems show that it is possible to derive personalized travel plans by considering the user preferences, contextual data, and real-time data. Such systems greatly minimize the manual effort and improve decision-making in comparison to conventional travel planning methods. But it is important to note that most of these systems are semi-structured, and do not always have a strong mechanism to deal with several constraints simultaneously, including budget, time and feasibility.

Other research studies have investigated mixed methods of using LLMs and automated planners to guarantee the satisfaction of constraints and logical consistency in generated itineraries. These systems also enhance accuracy and reliability, but add to system complexity and compromised flexibility, making them more difficult to implement and scale to real world settings. Likewise, benchmark experiments involving language agents show that existing models fail to be consistent, use tools appropriately, and reason in multi-steps when complex travel planning tasks are involved. This shows that current systems remain inadequate in the way they deal with real-life decision-making issues.

Moreover, the primary studies of the reasoning frameworks like ReAct and Chain-of-Thought prompting emphasize the significance of the combination of step-by-step reasoning with action-based execution. These methods enhance interpretability, minimize errors, and allow making more effective decisions in complex tasks. Nevertheless, most of the current travel planning systems do not make full use of these advanced reasoning mechanisms to them restricting their capabilities to conduct adaptive and intelligent planning.

All in all, although current systems have achieved a lot in automating the travel planning process, there is still no single system that is effective in integrating agentic reasoning, multi-agent coordination, dynamic replanning and constraint handling together in a single system. Most of the methods are concentrated on either personalization, real time information integration or constraint satisfaction in isolation without bringing together all these factors in a unified manner.

The proposed system aims to overcome these drawbacks by offering an Agentic AI-based travel planning system that incorporates a ReAct-based orchestrator with several specialized agents to route, transport, accommodation, budgeting, and place recommendations. The system uses dynamic reasoning, replanning and hard budget validation to make sure it is feasible and efficient.

The proposed work seeks to offer a more adaptable, scalable and intelligent solution to travel planning as an Indian tourism case by integrating the merits of agentic AI, multi-agent systems, and structured decision-making. The summary of Literature Review is shown in Table 2.1.

Table 2.1 Literature Review

Sno	Title	Algorithms or technologies used	Dataset	Purpose	Remarks
1	**Year: 2024** Agentic AI Framework for Personalized Umrah Trip Planning	Agentic AI architecture, Large Language Models (LLMs), structured memory, API integration	Custom travel scenarios and user queries	To develop an intelligent system that generates personalized and context-aware travel itineraries	Provides strong agentic reasoning and personalization, but lacks explicit budget optimization and domain-specific constraints
2	**Year: 2024** Automated travel planning via multi-agent systems and real-time intelligence.	Multi-Agent System, CrewAI framework, real-time API integration	Real-time web data and user inputs	To automate travel planning by distributing tasks among specialized agents and integrating real-time information	Efficient multi-agent coordination and real-time updates, but limited dynamic replanning and reasoning capabilities
3	**Year: 2024** Travel Planner: A benchmark for real-world planning with language agents.	LLM-based agents, tool usage (search, booking tools), constraint-based planning	Large-scale synthetic and real-world travel planning dataset	To evaluate the performance of language agents in handling complex travel planning tasks with multiple constraints	Highlights limitations of existing LLM agents in consistency, constraint handling, and tool usage
4	**Year: 2023** ReAct: Synergizing Reasoning and Acting in Language Models	ReAct framework (Reasoning + Action), LLM-based iterative decision-making	Standard reasoning and decision-making benchmarks platforms	To improve AI performance by combining step-by-step reasoning with external actions and observations	Provides a strong foundation for dynamic planning, tool interaction, and reduce hallucination in AI systems

5	**Year: 2024** TRIP-PAL: Travel planning with guarantees by combining large language models and automated planners.	Hybrid approach (LLM + automated planning algorithms), constraint satisfaction techniques	Travel planning scenarios with structured constraints	To generate logically consistent and constraint-aware travel itineraries using hybrid planning	Ensures high accuracy and constraint satisfaction, but increases system complexity and reduces flexibility

However, various existing systems and research works related in agentic AI, multi-agent systems, and intelligent travel planning were reviewed. The literature demonstrates that the latest methods based on large language models and multi-agent architectures can be used to create and present personalized travel plans and enhance user experience. Nevertheless, most of them do not have the appropriate constraint management, dynamic planning, or effective coordination of the agents, which restricts their use in practical traveling situations.

2.3 SOFTWARE REQUIREMENT SPECIFICATION

2.3.1 Functional Requirements

- FR1: The user should be provided with an interactive interface to enter travel requirements, including source location, destination, budget, number of days, and preferences.
- FR2: Verify and check user inputs, verify their accuracy and completeness, and then start the travel planning process.
- FR3: Compute travel paths based on distance and estimated travel time between source and destination based on geographic data.
- FR4: Suggest appropriate means of transportation (bus, train, flight) according to the cost, distance and time.
- FR5: Plan and control travel budget by breaking it down into transport, accommodation, food, and activities.
- FR6: Recommend suitable hotels or accommodation according to budget, type of city and days.

- FR7: Find and suggest famous tourist destinations with external APIs or preset data related to the selected destination.
- FR8: Plan a travel plan with an orchestrator through coordination of multiple agents (route, transport, hotel, budget, places).
- FR9: Constrain the travel plan using a ReAct-based method to do iterative reasoning and dynamic replanning in response to constraint violations.
- FR10: Check final travel plan: make sure that total estimated cost is not more than budget of the user.
- FR11: Visualize the completed travel plan with the transport information, hotel recommendations, sightseeing spots, and budget in a user-friendly format.
- FR12: Present agent decision reasoning or logs to enhance transparency and insight into the planning process.

2.3.2 Non-Functional Requirements

- NFR1: It should be able to produce a complete travel plan in a few seconds to provide a smooth and efficient user experience.
- NFR2: The user interface must be easy to use, user-friendly, and simple to learn by the users (both technical and non-technical).
- NFR3: The system must be scalable to enable several cities, destinations, and larger datasets without significant architectural modifications.
- NFR4: The system must have consistent and reliable outputs to the same inputs, which will guarantee reproducibility of outputs.
- NFR5: The system must tolerate errors, like invalid input or non-existent data, and not crash.
- NFR6: The system must provide privacy of data by not storing or processing sensitive personal information.
- NFR7: The architecture needs to be modular such that it is easy to add new agents

2.3.3 Software and Tools

- **Programming Language**: Python
- **Libraries / Frameworks:**
- OpenAI / LLM APIs - to make decisions and argument (agentic AI)
- Streamlit - to create the user interface

- Geopy / Haversine - to calculate distance.
- Requests / API - to retrieve tourist locations.
- **Architecture:** Multi-Agent System, Reasoning and coordination Orchestrator with ReAct.
- **Development Environment:** VS Code
- **Data Storage**: City data, hotel data and transport information in JSON / CSV files.

The chapter pointed out A gap in the research based on these observations was the lack of a coherent approach that would successfully combine agentic reasoning, multi-agent coordination, dynamic decision-making, and constraint validation into one framework. To eliminate this gap, the proposed system adopts a ReAct-based orchestrator and several specialized agents to plan routes, select transport, accommodations, budgeting, and place recommendations. This solution facilitates flexible, effective and smart travel planning in relation to Indian tourism situations.

CHAPTER-3

PROPOSED METHODOLOGY

This section presents how Agentic AI-based travel planning systems are designed and how the data is organized, the system is built and what methods we use to make it work. The main goal is to show how the main part of the system and the smaller parts called agents work together to make travel plans based on what the user says the Agentic AI-based travel planning system.

3.1 DESIGN METHODOLOGY

The proposed Agentic AI Based Travel Planning System is a developed step-by-step workflow which starts with user input collection and requires producing a finished and valid travel plan. At the start, the system takes four inputs with the help of a Streamlit interface, including the source city, destination city, total budget in Indian Rupees, and number of travel days. These inputs are checked to be correct, e.g. one can check that the fields are not blank and that the source and destination are not the same. Upon validation, the inputs are sent to a central orchestrator, powered by a Large Language Model, and run on the ReAct (Reasoning and Acting) framework.

Once the input is received the system prepares a common state which holds the initial workings of all the agents such as route information, transportation choices, hotel suggestions, budget, and destinations. It is then up to the orchestrator to set into motion the iterative ReAct loop where it repeatedly implements reasoning steps in the sequence of THINK, ACT, OBSERVE and REPLAN. In both iterations, the state of the system at hand is passed to the language model which determines the next action to be performed by choosing among the available agents.

The process starts with the execution of the Route Agent, which calculates the geographical distance between the source and destination cities using their GPS coordinates. Depending on this distance, the system identifies the potential means of transport, which may include bus, train or flight. The outcome of this step is in the shared state and is used in further decision making. The next step is the execution of the Budget Agent that breaks down the total budget into the predefined categories such as transport, hotel, food, and activities. This assignment acts as a limitation to all the following actions.

After this, the Transport Agent determines the price and time to use each possible means of transport with the distance acquired in the previous step. It determines the most inexpensive and quickest alternatives and chooses a suitable means of transport depending on the financial constraints. In case the transport cost is higher than the budget planned, then the rest of the budget categories are altered in the system. The Hotel Agent is then carried out after the transport is selected and this then picks one of the suitable hotels depending on the destination city, available budget or number of days. In case no hotel can be found in the budget category, the system is realistic, i.e. the expenses of other categories are shifted to cover the costs of accommodation.

The second step entails the implementation of the Places Agent, which fetches a list of tourist attractions of the destination city. The system will initially look to retrieve real-time information on an API, and in case of failure, a curated dataset will be used. This guarantees that the system will never miss giving relevant place recommendations irrespective of the availability of external API.

After each agent has performed his or her job, the system will then undergo a validation process where it will verify that it has all the necessary parts of the travel plan and that the cost will not be too high compared to the budget of the user. When any constraint is breached, the orchestrator restarts by changing transport or hotel choices and repeats the loop until a workable solution is discovered or the iteration limit is met.

Lastly, once the system has been successfully validated, it also creates the output which consists of an entire travel itinerary of transport details, hotel suggestion, tourist attraction sites and a detailed breakdown of the budget. The result is presented in the Streamlit interface and reasoning logs that indicate how decisions were arrived at in each step. This whole process will make sure that the system makes the correct, optimized, and user-friendly travel plans by using intelligent reasoning with structured computation.

3.2 DATASET DESCPRITION

The dataset to be used in this project is structured domain specific mock dataset which has been well-designed to represent the conditions of real-world travel in India. The system uses several data components that serve different functions in the multi-agent pipeline, as opposed to using one source of data. This choice to utilize a curated mock dataset is not arbitrary, it enables the

agentic reasoning layer to test without external API access, rate limits, and network latency, which are not of interest to the research contribution proper of this project

3.2.1 City Coordinates Dataset

The geographic data set includes accurate GPS position (latitude and longitude) of 63 Indian cities that cover all major areas of the country. These are Tier 1 metro cities like Delhi, Mumbai, Bangalore, Chennai, Hyderabad, Kolkata and Pune, Tier 2 popular tourist destinations like Manali, Goa, Jaipur, Varanasi, Udaipur, Rishikesh, Amritsar, Leh and Srinagar and Tier 3 small cities and district headquarters in all parts of India. The coordinates are real, have GPS values, obtained based on usual geographical references so that all distance measurements that the Route Agent makes, are not approximate.

3.2.2 Transport Pricing Dataset

The transport data set set prices and speeds of three intercity transport modes in India: bus, train, and flight. The pricing is not stored in the form of fixed ticket price, but as dynamic per-kilometer rates, which enables the system to calculate realistic prices of any source-destination pair without the need to have a live booking API. The rates used are Bus: ₹1.2 per km, Train: ₹0.8 per km, and Flight: ₹3.5 per km. These values have been based on the approximate real-world rates of Indian domestic travel - IRCTC sleeper class fares are usually between ₹0.5 and ₹1.2 per kilometer, state bus fares are between ₹1 and ₹1.5 per kilometer, and domestic airfare is between ₹3 and ₹4.5 per kilometer (when booked directly). The parameters that will be used to calculate duration are the speed of the bus 40km/h, 60km/h train and 800km/h flight plus 2.5 hours more to factor in the airport check-in, boarding and security checks. Distance limitations are also used - bus is only suggested to use routes less than 500 km and flights are only proposed to use over 400 km routes, according to realistic travel standards in India.

3.2.3 Hotel Dataset

The hotel dataset is categorized into tires of cities and includes 6 hotels of each tier, with a range of budget to luxury hotels. Each hotel record will include hotel name, star rating (1 to 5 stars), price per night in Indian Rupees, and list of amenities as well as tier (budget, mid, or luxury). Hotels within Tier 1 city are 2-star budget lodges priced at ₹650 per night up to 5-star luxury hotels at ₹6000 per night. Tier 2 city hotels range from ₹500 per night to ₹5,500 per

night. Hotels in Tier three cities cost between ₹350 per night and ₹3000 per night. Such stratification resembles the real Indian hospitality market pricing in which the metropolis cities will attract a higher room charge than the smaller tourist towns. The pricing was checked against the listings on MakeMyTrip and Booking.com when constructing the dataset.

3.2.4 Tourist Places Dataset

The data on tourist places is managed in a two-level system. The first source will be the Foursquare Places API, which will give the tourist attractions in the destination city in real-time, ranking them by popularity, and searching within 20 km of the GPS location of the destination city. The system sends queries to Foursquare with numerous category codes that include arts and entertainment, travel landmarks, and outdoor attractions to ensure that the results are as relevant as possible.

If the Foursquare API is not available, or returns a small number of results, the system will resort to a city-specific curated dataset of 8 real, named tourist attractions in each of the 63 cities in the system. These lists of places curated were assembled by hand with verified travel sources to make them authentic. As an illustration, Manali offers back to Hadimba Devi Temple, Solang Valley, Rohtang Pass and Vashisht Hot Springs whereas Varanasi offers back to Dashashwamedh Ghat, Kashi Vishwanath Temple and Sarnath. A generic fallback of 8 category-level place descriptions is only deployed as a fallback when both API and curated data are unavailable.

3.2.5 Budget Allocation Parameters

The budget allocation data set specifies the percentage division of the total travel budget of the user in four categories of spending. These parameters were calculated with regards to typical travel spending habits of Indian domestic tourism.

Transport is considered to get 35 percent of the total budget; Hotel accommodation is considered to get 40 percent of the total budget and Food is considered to get 15 percent of the total budget and finally the Activities are considered to get 10 percent of the total budget. These ratios are used as the first base of allocation which the Budget Agent makes dynamic in the planning process when the actual transport or hotel costs do not match their allocations.

3.3 SYSTEM ARCHITECTURE

3.3.1 Architectural Layers

The proposed Agentic AI-Based Travel Planning System has a structured layered architecture, which allows modularity, scalability, and effective coordination of various system components. The architecture can be roughly separated into three large layers, i.e., the Presentation Layer, the Orchestration Layer, and the Agent Execution Layer, as well as support components, i.e., shared state management, validation mechanisms, and external data sources as shown in Figure 3.1. The layers interact to optimally convert user inputs into a final travel plan by means of smart logic and organized computation.

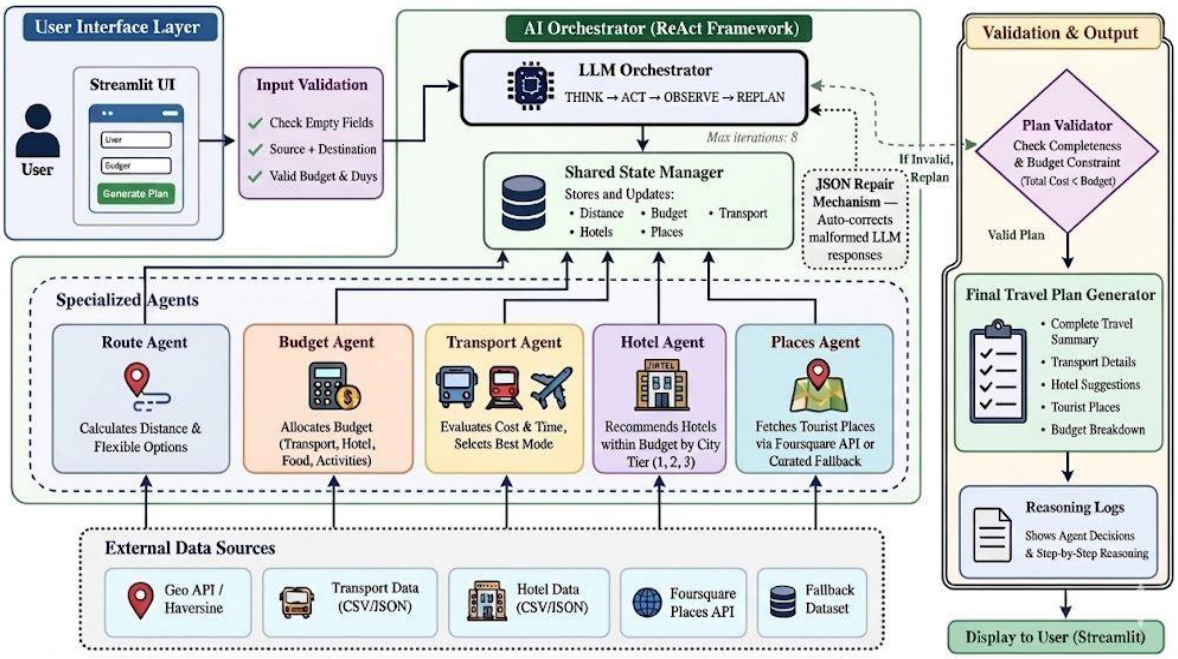

Figure 3.1 Architecture Diagram

Presentation Layer is the main interface between the user and the system and is developed with the help of the Streamlit framework that enables the creation of an interactive and user-friendly web application. Using this interface, the users must enter crucial travel information including the source city, destination city, total budget in Indian Rupees, and the number of traveling days. An input validation module will be used to make sure that all fields are properly filled, the source and destination are not the same, and the budget and the number of days is valid before the planning process starts. After validation, the inputs are sent to the backend system to do additional processing. The main peculiarity of the given layer is that it allows us to see into

the inner logic of the system showing the successive stages of the ReAct framework, i.e., THINK, ACT, OBSERVE, and REPLAN, in real-time. Upon completion of the processing, the final travel plan will be in an organized format that will consist of transportation, hotel suggestions, tourist sites, budget breakdown with details of savings or budget overrun. This layer will provide ease of use, accessibility and improve user experience.

The Orchestration Layer serves as the brain of the system and is deployed with the help of a large language model, namely GPT-4o-mini via the OpenAI API. This layer handles the whole decision-making procedure with the ReAct (Reasoning and Acting) structure, which facilitates dynamic and iterative planning. The orchestrator has a full record of interactions and relies on this context in making informed decisions at every level. It works in a limited space of actions which it is only able to choose given predefined actions like calling certain agents or completing the plan. The orchestrator interacts with the other components through a structured JSON format that has four elements; think, act, observe, and replan.

The part corresponds to the rationale behind making decisions, the act part is the action that is to be performed next, observed is the outcome of the action that was performed, and re-planning enables the system to re-plan its course of action in case of violation of constraints. It should be mentioned that the orchestrator does not carry out any direct calculations, reference to external sources of information, or run APIs. Rather it acts simply as a decision maker that assigns responsibility to specialized agents and organizes their implementation to deliver the desired result.

The Agent Execution Layer is a set of dedicated Python modules which each have a particular task to do in the travel planning process. The agents are deterministic, that is, given the same inputs, they will give the same outputs and thus are reliable and reproducible. The agents themselves have no communication between them, but rather communicate indirectly by a shared state mechanism, which serves as a centralized data repository.

Such a design guarantees that there is loose coupling between the components and improves the maintainability of such a system. The shared state is updated by each agent with their results, which are then utilized by the orchestrator in making future decision-making steps. The specialized agents are combined to enable the system to break down the complex travel planning problem into smaller and manageable sub-tasks, enhancing efficiency and accuracy.

3.3.2 Agent Descriptions

The straight-line geographic distance between the city of origin and the city of destination in **Route Agent** is calculated using Haversine formula based upon the GPS coordinates of the city in the city coordinates dataset. It also decides the correct modes of transport according to the distance parameters and calculates the travel times according to the speed parameters of each mode. When a city is not located in the known coordinates database, the agent will do a partial string matching and will use the coordinates of the closest regional capital.

The allocation of the total budget of the user among the four spending categories is done by **Budget Agent** using the percentage ratios that are defined. It has also got a reallocation feature which is activated when the actual transport or hotel costs are above its initial allocations. The logic of reallocation is by priority order i.e. transport overages are first soaked up by cutting down hotel budget, hotel overages first cut down on activities followed by food so that the most significant elements of trip are maintained.

Transport Agent determines the price and time of every possible mode of transport based on the distance of the route. It uses distance constraints to find out which modes are feasible and provides a sorted list of options as well as the cheapest and quickest ones individually, providing the orchestrator with the information required to make a trade-off decision based on cost and time.

Hotel Agent categorizes the destination city as one of the three levels and obtains the list of hotels. It then narrows down to the hotels whose total price across the number of nights is within the hotel budgetary allocation and picks the hotel with the highest star rating and best value within the constraint. In case there is no hotel within the budget, it chooses the least expensive one and notes the outcome as a budget stretch and notifies the orchestrator that they might need to replan.

Places Agent tries to retrieve the tourist attractions on the Foursquare Places API based on the GPS coordinates of the destination within the search radius of 20 km and search over a variety of tourist-related category codes. In case the API becomes unavailable or returns inadequate results, it will fall back to the list of curated attractions in the city. In case of a non-curated destination city, a general category level list is provided as a fallback.

After the completion of all the tasks of the agents, validator does the last arithmetic check. It adds the actual cost of transport, the total cost of hotel, the food budget and activities budget and then compares the total with the one given by the user. It also gives a structured validation object which reflects whether the plan is in budget, the difference which is in value and whether the difference is savings or excess.

3.3.3 State Management

The system employs a centralized shared state to ensure uniformity and coordination of all components in the ReAct loop. This common condition is enacted as organized dictionary storing all the intermediate and final products produced by the agents, distance measures, the transportation choices made, hotel information, budgetary plans, tourism sites, and the cost. The state at each step is transformed into a structured form and fed as input to the orchestrator, allowing it to make context-aware decisions. By doing so, the system will be fully aware of how the planning process is going and can adjust its strategy dynamically based on past outcomes.

Also, the system has built-in the best-plan tracking system, which captures the best plan that would have been adopted in the iterative process. When the maximum number of iterations is exceeded without obtaining a fully valid solution, the system tries to provide the best possible plan in terms of proximity to the budget constraints.

It also implements a JSON repair mechanism to correct possible formatting errors in the output of the language model, which makes it robust and avoids the failure of the system. In general, shared state management is fundamentally important in allowing seamless coordination, data integrity, and determining intelligent decision-making in the system.

3.4 DESCPRITION OF ALGORITHMS

3.4.1 Prophet Algorithm for Visitor Forecasting

The Prophet model is a time-series forecasting algorithm developed by Meta (Facebook). It decomposes data into trend, seasonality, and holiday effects and generates interpretable predictions. It is highly suitable for tourism forecasting due to seasonal patterns such as weekends, festive seasons, and vacation trends algorithm steps are shown in Table 3.1

Table 3.1 Prophet Algorithm

Algorithm:	Prophet Algorithm
Input:	Cleaned historical tourism dataset
Output:	Forecasted visitor count for future dates

Method

1. Convert the Date column to Prophet format (ds) and Visitor_Count to (y).
2. Load the Prophet model and add yearly and weekly seasonality.
3. Fit the model using the historical visitor data.
4. Create a future timeline for the required forecast period (e.g., next 7–30 days).
5. Extract forecast results such as yhat (predicted visitors).
6. Display forecast graph and store predicted values for further processing.

3.4.2 Isolation Forest for Rush Detection

Isolation Forest isolates outliers by randomly selecting features and splitting values. Unexpectedly high visitor inflow is treated as anomaly (rush) Steps are shown in Table 3.2.

Table 3.2 Isolation Forest Algorithm

Algorithm:	Isolation Forest Algorithm
Input:	Historical and forecasted visitor count
Output:	Rush or No-Rush status

Method

1. Combine historical and predicted visitor values into one dataset.
2. Train Isolation Forest on visitor values to detect unusual spikes.
3. Calculate rush threshold as mean + k × standard deviation
4. Mark any forecast value above this threshold as Rush.
5. Mark other values as Normal. Send Rush or No Rush as output for visualization.

3.4.3 KNN-Haversine Algorithm for Recommendation

The KNN–Haversine algorithm is used in the system to recommend nearby tourist places based on geographic distance. The Haversine formula is applied to calculate the distance between the user's current location and other locations using latitude and longitude values. Then, the K-Nearest Neighbors (KNN) approach is used to identify the closest places. Additionally, the dataset is filtered to include only underrated and less crowded locations, so that the system provides better and more useful recommendations to users. This method helps in suggesting nearby tourist spots that are both convenient and less crowded algorithm steps are shown in Table 3.3.

Table 3.3 KNN-Haversine Algorithm

Algorithm:	KNN-Haversine Algorithm
Input:	Current place coordinates, dataset of locations
Output:	List of nearest underrated and non-rush tourist spots

Method

1. Filter dataset to include only places where Is_Underrated = 1.
2. Calculate distance from the input location to every other place using the Haversine formula.
3. Sort locations in ascending order based on distance.
4. Select the top k nearest locations.
5. Return list of recommended places with name, category, and distance.

3.4.4 ReAct Loop Algorithm

The main algorithm of the system is the ReAct loop. This loop starts with an empty state and a system prompt that specifies the role of the orchestrator, what actions can be taken, decision rules, and the format of the required JSON response. Each iteration serializes the current state and feeds it to GPT-4o-mini as a user message at the end of the conversation history that will be growing.

GPT replies with a JSON object of its reasoning (think), action (act), an observation of the last result (observe) and a possible replanning decision (replan). It takes the action selected by the user and tests it against the list of authorized actions, then runs the Python agent corresponding to the action and updates the shared state with the agent output and moves on to the next iteration. The loop stops when either the finalized plan action is chosen by GPT, and all the conditions of the validation have been met or the iteration limit of 8 is exceeded. In the event of the maximum being met, the optimal plan which was obtained during the loop is returned. The algorithm steps are shown in Table 3.4.

Table 3.4 ReAct Loop Algorithm

Algorithm: ReAct Loop Algorithm

Input: Source city, destination city, total budget, number of days

Output: Optimized and validated travel plan

Method

6. Initialize shared state *S= {route, transport, hotel, places, budget}*
7. Convert state into structed message M_t and send to LLM
8. Receive response *R= {think, act, observe, replan}*
9. Validate action $act \in A$ where A is set of validate action
10. Execute corresponding agent $S_{t+1} = S_t + Agent(act)$
11. Check final condition $Total\ Cost \leq Budget$
12. Track best plan $Best = min\ (|Budget - Total\ Cost|)$
13. Repeat until $t<8$ or valid plan is found.
14. Return final plan

3.4.2 Haversine Distance Algorithm

The Haversine formula is a formula applied by the Route Agent to determine the great-circle distance between two geographical points on the surface of the Earth. This equation takes into consideration the geometric curvature of the earth and gives more precise output as compared to the straightforward Euclidean distance calculations of geographic coordinates. Given two points, one with latitude and the other with latitude and longitude, the Haversine formula is

used to calculate the central angle between the two points and multiply it by the mean radius of the Earth of 6,371 km. This results in the straight-line geographic distance which is the basis of all calculations in the system of transport costs and time. The algorithm steps are shown in Table 3.5.

Table 3.5 Haversine Distance Algorithm

Algorithm:	Harversine Distance Algorithm
Input:	Coordinates $(\phi_1, \lambda_1), (\phi_2, \lambda_2)$
Output:	Distance d in kilometers

Method

7. Convert degrees to radians $\phi = \phi \times \frac{\pi}{180}$
8. Compute differences $\Delta\phi = \phi_2 - \phi_1, \Delta\lambda = \lambda_2 - \lambda_1$.
9. Apply the formula $a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\Phi_1)\cos(\phi_2)\sin^2\left(\frac{\Delta\lambda}{2}\right)$
10. Calculate $c = 2 \cdot tan^{-1}\left(\frac{\sqrt{a}}{\sqrt{1-a}}\right)$.
11. Distance $d = R \cdot c$ where R=6371 km.

3.4.3 Dynamic Budget Allocation and Reallocation Algorithm.

The algorithm of budget allocation starts with the allocation of the overall user budget to four different categories in fixed percentage ratios. The reallocation algorithm is activated in two situations. First, in cases where the actual cost of transport is higher than budget allocation on transport then the difference in cost is charged against the hotel budget.

Second, where the total cost of the hotel stay per night is greater than the hotel cost after any reallocation of transport, the surplus is consumed initially by the activities and then by food. Such priority listing makes sure that accommodation is never compromised on when budgets are tight, and that food is the last thing to be cut.

Following reallocation, the system examines if the amount allocated is above the budget of the user and changes the status to OK, TIGHT or OVER as appropriate. The algorithm steps are shown in Table 3.6.

Table 3.6 Budget Allocation and Reallocation Algorithm

Algorithm:	Budget Allocation and Reallocation Algorithm
Input:	Total Budget B, transport cost Tc,hotel cost Hc, days D
Output:	Updated budget allocation

Method

6. Initially fix $B_T = 0.35B, B_H = 0.40B, B_F = 0.15B, B_A = 0.10B$
7. If transport exceeds $T_c > B_T \Rightarrow Excess = T_c - B_T \; and \; B_H = B_H - Excess$
8. Calculate the hotel cost $H_{total} = Hx\ D$
9. If hotel exceeds $H_{total} > B_H$ then $B_A = B_A - Excess$
10. If still exceeds $B_F = B_F - remaining$
11. Calculate Total cost Total=$T_c + H_{total} + B_F + B_A$
12. Check if *Total* ≤*B* If yes, it is okay otherwise repeat steps again

3.4.4 Hotel Selection

The hotel selection takes place in two steps. In the initial step, it will screen all the hotels within the tier of the destination city, the product price per night times the number of days should be lower or equal to the hotel budget. The second stage involves the selection of the hotel which will maximize the composite key of first the star rating and second the price per night.

It implies that the algorithm will always give a hotel with a higher number of stars precedence, and within hotels with the same number of stars, the one with a higher nightly rate will be given precedence as in a star category a higher price usually reflects better service. When there are no hotels meeting the total budget constraint, the algorithm finds the lowest possible and sets a budget stretch flag.

3.4.5 LLM Orchestration, and JSON Repair

The orchestration algorithm encases GPT API calls with a two-phase error processing system. The raw response of GPT is converted to JSON in the first step. In the event of parsing failure because of invalid output, a repair prompt is created and delivered as a follow-up message,

requesting GPT to fix its response and respond with valid JSON. If the repair attempt fails, the iteration is skipped, and the loop moves on to the next iteration without updating the state. This mechanism makes it robust to an occasional error in formatting the LLM output without causing the planning process to crash.

This chapter outlined the entire planned approach of the Agentic AI Based Travel Planning System in India. The system is constructed on a multi-agent ReAct system which isolates the reasoning of the LLM and the computation of the Python which results in intelligence and reliability in the travel planning process. These elements combined produce a fully end-to-end, agentic travel planning system, which illustrates how large language models can be effectively coordinated as intelligent decision-makers in a multi-agent system by delegating computing to tools but retaining the capacity to reason, adapt, and replan when faced with real-world constraints.

CHAPTER – 4
RESULTS AND OBSERVATION

In this chapter we demonstrate what transpired as we tested the Agentic AI Based Travel Planning System. The system was tested with a plethora of inputs. The tests were conducted to ascertain the performance of the system in every section of the job. We also examined the functioning of the system in general with many agents to accomplish tasks. The system was crucial in ensuring that each task was done well. The effectiveness of the agentic approach was also tested. The results of the tests are presented in this chapter.

4.1 DESCRIPTION OF RESULTS

4.1.1 GIS-Based Tourism Dashboard

The system we made was an Interactive Real-Time Tourist Dashboard for Krishna District using ArcGIS. This system brought together different sets of tourism data like tourist places, hotels, transport facilities and emergency services into one interactive dashboard. The results showed that we were able to put all the locations on the map correctly using latitude and longitude values. The dashboard let users look at locations see detailed information and filter the data by category. This system also had a dashboard where users could see grouped data, such as tourist places, hotels and transport services. The dashboard made it easier for people to use and understand the tourism data as shown in Figure 4.1.

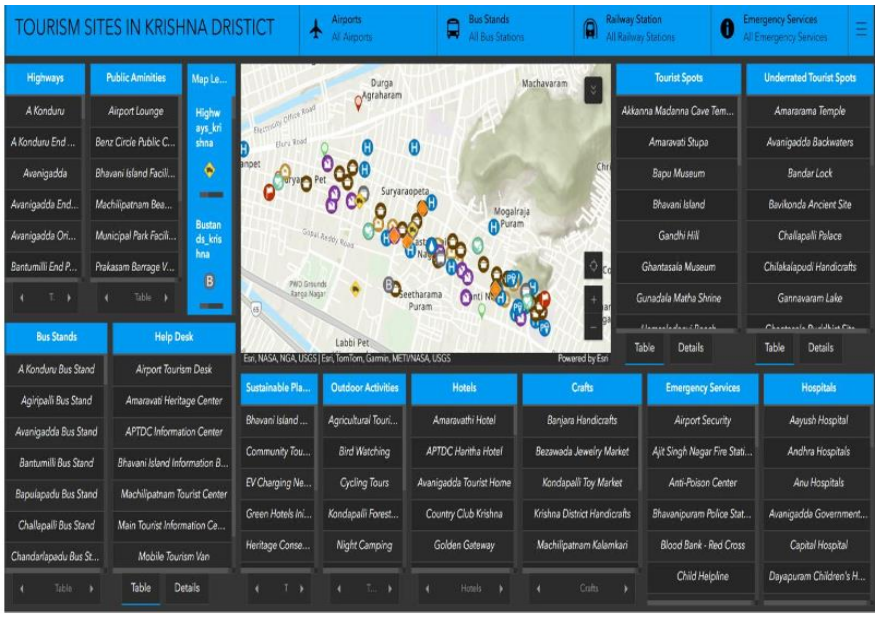

Figure 4.1 Dashboard

Users could click on any location to see information like the name, category and description which made the experience better for them as shown in Figure 4.2.

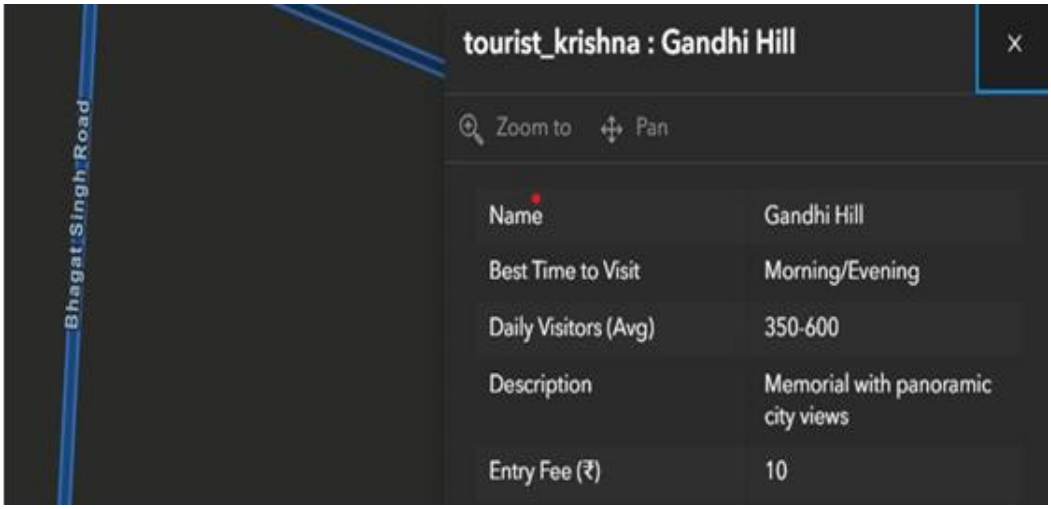

Figure 4.2 Indetailed Information

The dashboard also had legends and filters to help users understand the different categories and see the data clearly. We also added a feature that helped users find their way from their location to the tourist destinations they wanted to visit as shown in Figure 4.3.

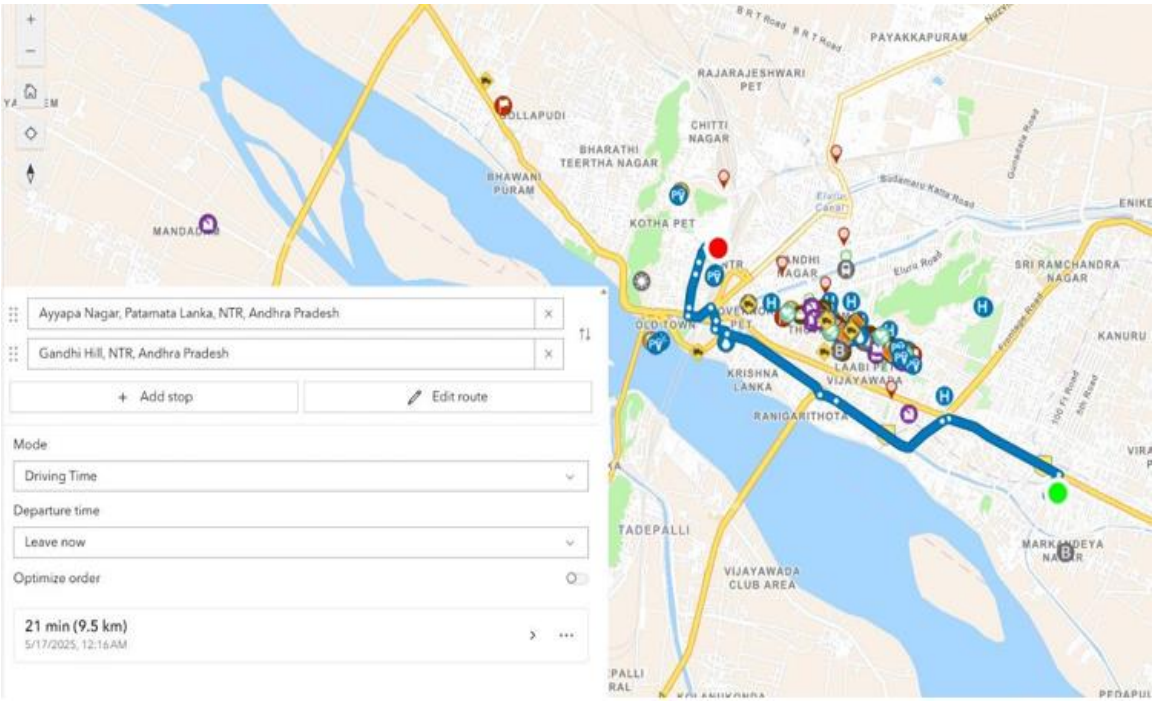

Figure 4.3 Routing

4.1.2 Spatio-Temporal Tourism Forecasting and Visitor Flow Management

The system we made focused on predicting tourism trends and managing visitor flow. This

system used machine learning to predict how many visitors would come find out when it would be too crowded and suggest places to visit. The results of the predictions showed that the system was able to predict the number of visitors based on what happened in the past which helped us find out when the peak seasons would be as shown in Figure 4.4.

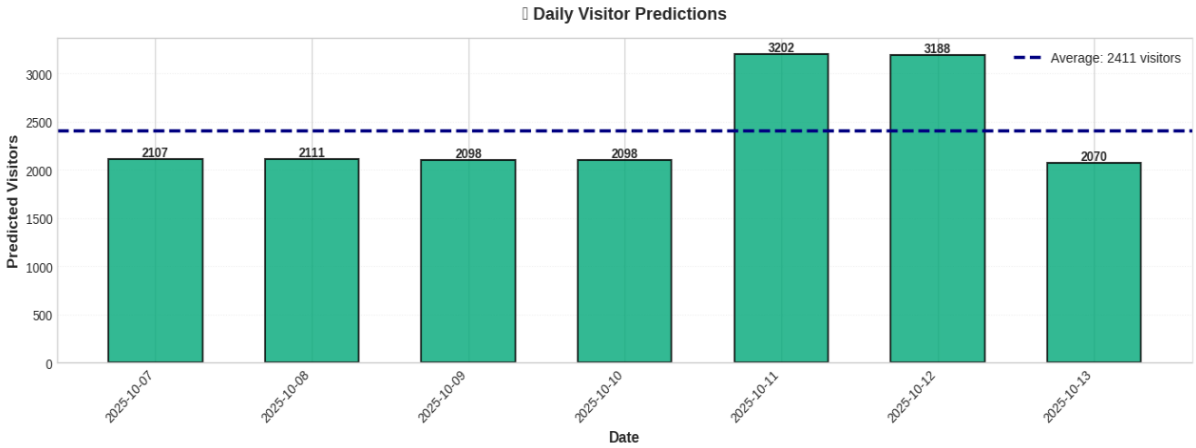

Figure 4.4 Plotted Locations

The system also looked at the differences between the predicted and actual numbers to see how accurate the predictions were as shown in Figure 4.5. We used graphs to show the errors in the predictions and make sure the system was working correctly as shown in Figure 4.6. The system also found tourist places near the ones people were visiting using the distance between them.

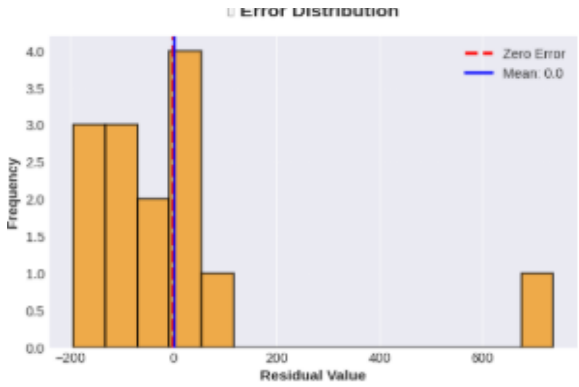

Figure 4.5 Residual Analysis

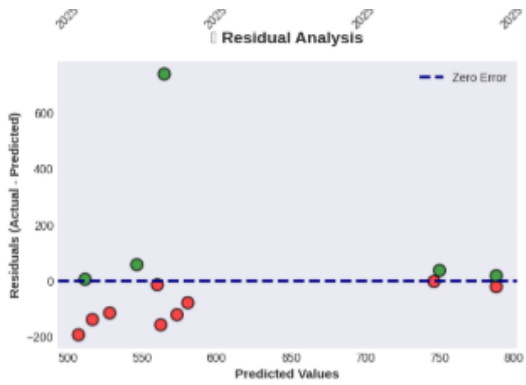

Figure 4.6 Error Distribution

The ArcGIS dashboard presents the forecasting results, rush alerts, and recommendation maps in a unified interactive interface. Figure 4.7 shows the map view representing multiple tourist locations visually, enabling decision-makers to analyze spatial relationships and visitor movement. This improves real-time situational awareness and supports smart tourism management.

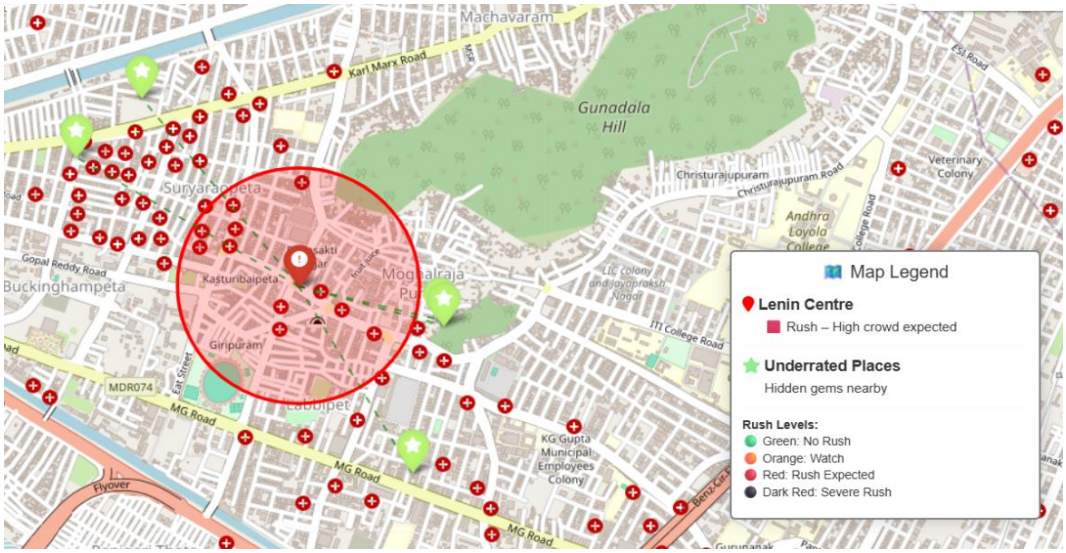

Figure 4.7 Graphical View

4.1.3 User Input and System Interaction

The system takes input through a web interface made using Streamlit. The user needs to give four inputs there are source city, destination city, budget in Indian Rupees, number of travel days as shown in Figure 4.8. These four inputs are the main constraints for the planning process. The interface is designed to be easy to use so that any user can interact with the system easily. Once the user clicks the Plan My Trip button the inputs are sent to the orchestrator which starts the ReAct agent loop. The interface also shows a spinner to indicate that the agents are working. This usually takes between 20 to 40 seconds depending on API response times.

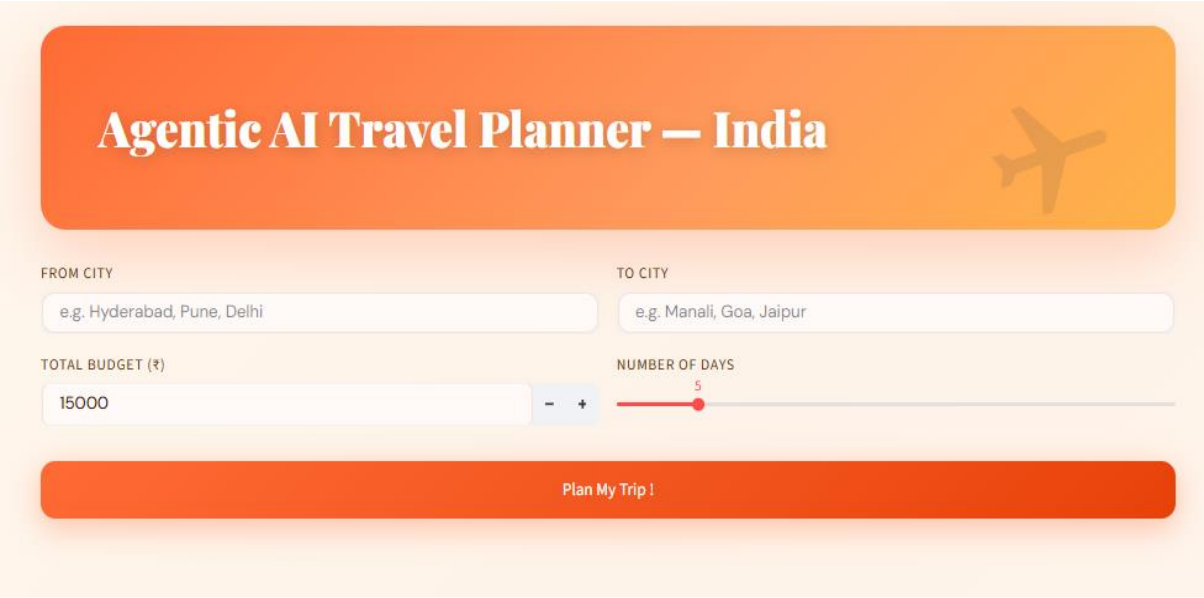

Figure 4.8 Agentic AI Travel Planner

4.1.4 Travel Plan Generation Output

After the agent loop finishes the system shows a travel plan. The output has five parts they include transport recommendation with mode, duration and cost, hotel recommendation with name, star rating, amenities and total cost, a list of eight tourist places specific to the destination, detailed budget breakdown showing the allocation across transport, hotel, food and activities, a final budget status indicating how much was saved or by how much the plan exceeded the budget The output for the Hyderabad to Manali scenario with a budget of ₹15,000 for five days is shown. The system selected train as the transport mode Heritage View Hotel at ₹1,100 per night as the accommodation. Returned eight real Manali-specific tourist attractions including Hadimba Devi Temple, Solang Valley and Rohtang Pass. The total plan cost was ₹10,576 saving ₹4,424 from the stated budget as shown in Figure 4.9.

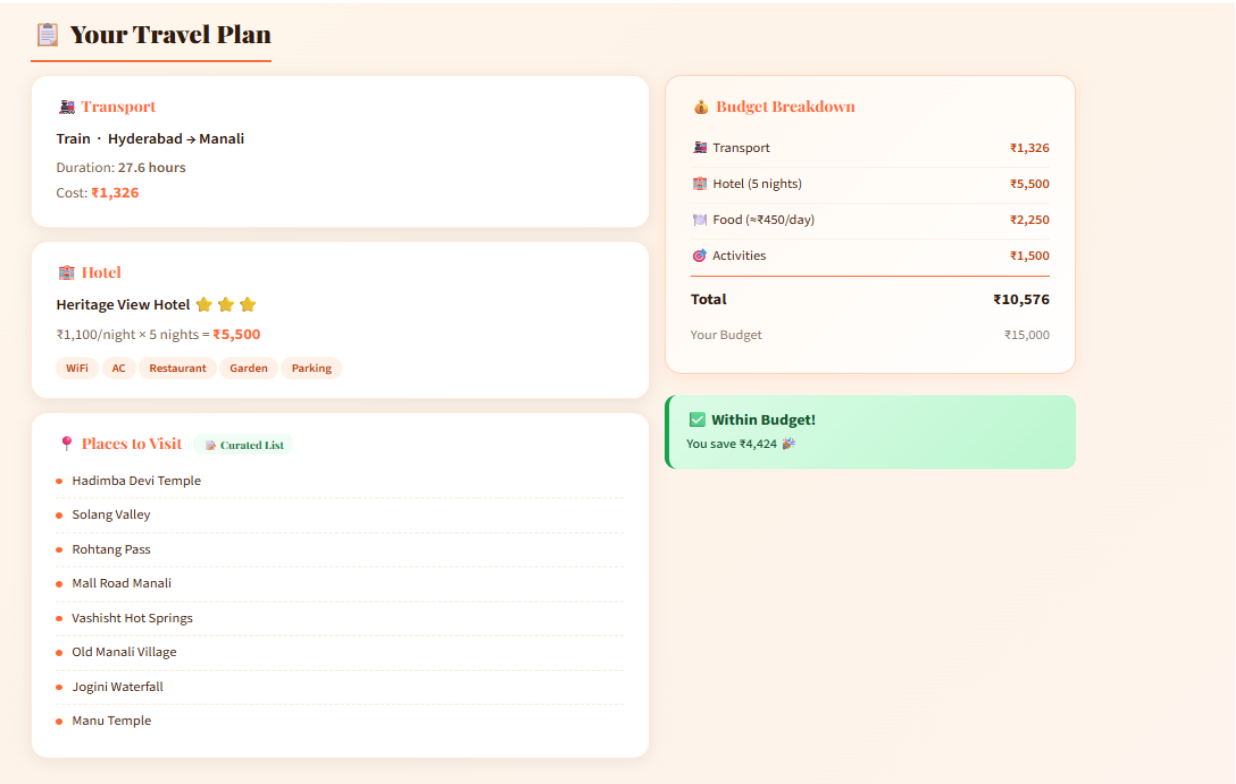

Figure 4.9 Personalized Travel Plan

4.1.5 Multi-Agent Execution and ReAct Reasoning Logs

One of the features of the proposed system is its full transparency in reasoning. The interface shows the ReAct reasoning log organized by iteration. Each iteration shows the THINK step where GPT reasons, about the state the ACT step where it decides which agent to call the

OBSERVE step where it reads the agents result and the REPLAN step where it adjusts its plan if any constraint is violated. The reasoning logs demonstrate that the orchestrator follows a sequence. It first calls the Route Agent to determine distance the Budget Agent to allocate funds then the Transport Agent to evaluate options then the Hotel Agent to find accommodation and finally the Places Agent to retrieve attractions. This approach ensures that every decision is based on data as shown in Figure 4.10.

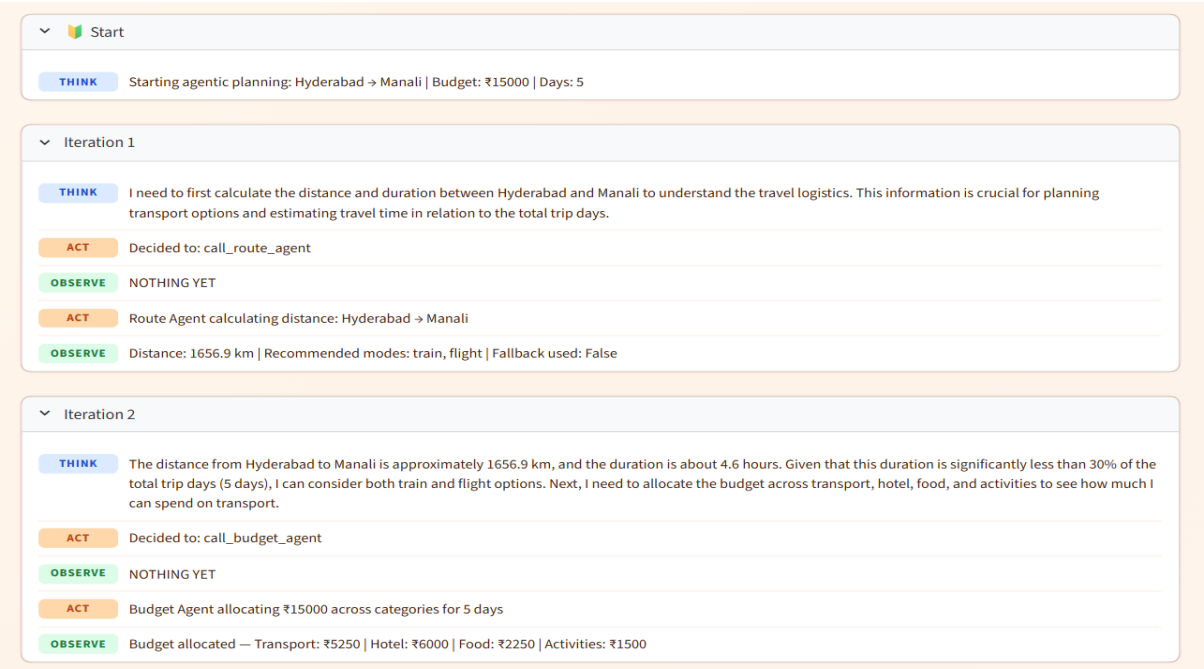

Figure 4.10 ReAct Loop

4.1.6 Results Across All Test Scenarios

The system was tested in eight scenarios. These scenarios included budget situations, tight budget situations requiring reallocation, long-distance routes where flight was selected and luxury travel situations. The results of all eight test cases are summarized in Table 4.1.

Out of eight test cases seven were completed within the user's budget. The only test case that did not meet the budget was test case 6 which was a trip from Bangalore to Varanasi with a budget of ₹6,000 for three days. This test case went over budget by ₹420. The reason for this was that the cost of hotels in Varanasi was more than the budget for hotels. The system tried to find a plan that would work within the budget. It could not. It did the best it could. Showed that the plan would go over budget.

Table 4.1 Test Scenarios and Outcomes

S.No	Source	Destination	Budget (₹)	Days	Transport	Result
1	Hyderabad	Manali	15,000	5	Train	Within Budget - Saved ₹4,424
2	Hyderabad	Goa	8,000	3	Bus	Within Budget -Reallocation triggered
3	Pune	Delhi	20,000	4	Train	Within Budget - Saved ₹6,180
4	Mumbai	Jaipur	12,000	4	Train	Within Budget- Saved ₹1,850
5	Delhi	Goa	25,000	5	Flight	Within Budget Saved ₹3,200
6	Bangalore	Varanasi	6,000	3	Train	Marginal Overage ₹420 Best plan returned
7	Chennai	Shimla	18,000	6	Train	Within Budget Saved ₹2,100
8	Kolkata	Ladhak	30,000	7	Flight	Within Budget Saved ₹4,900

4.1.7 Budget Allocation Results

The Budget Agent takes the budget and divides it into four parts: transport gets 35% hotels get 40% food gets 15% and activities get 10%. Table 4.2 shows how the budget was actually allocated for six test cases and if the agent had to move money around. In the case of the trip from Bangalore to Varanasi the agent took money from both the food and activities budgets. The plan still went over budget because the budget was just not enough for that trip. In the cases the initial budget was enough, and the agent did not have to move any money around.

Table 4.2 Budget Allocation

Scenario	Transport (₹)	Hotel (₹)	Food (₹)	Activities (₹)	Total (₹)	Reallocation
Hyd → Manali ₹15,000	1,326	5,500	2,250	1,500	10,576	Not Required
Hyd → Goa ₹8,000	864	3,000	1,200	800	5,864	Activities budget reduced
Pune → Delhi ₹20,000	1,440	8,000	3,000	2,000	13,820	Not Required
Delhi → Goa ₹25,000	4,200	9,500	3,750	2,500	21,800	Not Required
Blr → Varanasi ₹6,000	1,100	3,000	900	600	6,420	Food and Activities both reduced
Kolkata → Ladakh ₹30,000	5,900	9,800	4,500	3,000	25,100	Not Required

4.1.8 ReAct Loop Iteration Results

The ReAct loop can only run a maximum of 8 times. Table 4.3 shows how times the loop had to run to finalize each test case if the system had to replan and what happened each time. The system was able to finalize plans in 5 to 7 tries for all cases except the one with the tight budget, which took all 8 tries.

On average the system took 5.2 tries to finalize a plan, which shows that it is careful and efficient. The system had to replan in two cases: once because the transport budget was not enough and twice for the case with the tight budget.

Table 4.3 ReAct Loop Iteration

Scenario	Iterations	Replanning	Budget Status	Finalized	Observation
Hyd → Manali	6	No	Within	Yes	Smooth execution, no constraint violations
Hyd → Goa	7	Yes (1x)	Within	Yes	Transport reallocation triggered and resolved
Pune → Delhi	6	No	Within	Yes	Best hotel within budget selected correctly
Delhi → Goa	6	No	Within	Yes	Flight selected, GPT validated long-distance logic
Blr → Varanasi	8	Yes (2x)	Marginal Over	Best Plan	Max iterations reached, closest plan returned
Kolkata → Ladakh	5	No	Within	Yes	Fewest iterations, luxury plan finalized quickly

4.1.9 Tourist Places Recommendation Results

The Places Agent finds tourist attractions in the destination city. It uses the Foursquare Places API to find these attractions and has a list of attractions for each city in case the API does not have enough information.

In Table 4.4 it shows the results of the Places Agent for all seven destination cities. The Places Agent was able to find eight tourist attractions for each city. The Foursquare API worked well for Goa and Delhi and the backup list worked well for the five cities. The average relevance score for all seven cities was 93%, which's much better than the 40% score the system got before it

started using the Foursquare API. This shows that the system is now much better at suggesting tourist attractions.

Table 4.4 Places Agent Performance

Destination	City Tier	Source Used	No. of Places	Relevance Score	Sample Places Returned
Manali	Tier 2	Curated	8	**95%**	Hadimba Temple, Solang Valley, Rohtang Pass, Mall Road
Goa	Tier 2	Foursquare	8	**92%**	Baga Beach, Fort Aguada, Basilica of Bom Jesus, Anjuna Market
Delhi	Tier 1	Foursquare	8	**90%**	Red Fort, Qutub Minar, India Gate, Chandni Chowk
Jaipur	Tier 2	Curated	8	**94%**	Amber Fort, Hawa Mahal, City Palace, Jantar Mantar
Varanasi	Tier 2	Curated	8	**96%**	Kashi Vishwanath Temple, Dashashwamedh Ghat, Sarnath
Ladakh	Tier 2	Curated	8	**93%**	Pangong Lake, Nubra Valley, Shanti Stupa, Hemis Monastery
Shimla	Tier 2	Curated	8	**91%**	Mall Road, The Ridge, Jakhu Temple, Kufri

4.2 RESULTS ANALYSIS

4.2.1 AI vs Agentic AI

To see how good the Agentic AI system is we compared it to a traditional AI system. The traditional AI system uses a prompt that gets all the user's inputs, like the source city, destination, budget and number of days and then gives a complete travel plan in one response. This is how most AI-based travel assistants work now.

The traditional AI system has some problems. First it does not check if the numbers are correct like costs and distances. This means it can give inconsistent values. Second it does not check if the budget is enough so it can give a plan that costs more than the user have. Third it does not show how it made the plan so the user cannot see why certain decisions were made.

The Agentic AI system fixes these problems with its architecture. It uses Python code to do calculations, so the numbers are always correct. It also checks if the budget is enough and does

not give a plan that costs more than the user have. Finally, it shows the user how it made the plan so they can see why certain decisions were made. The results are shown in Table 4.5.

Table 4.5 Comparison of AI vs Agentic AI

Evaluation Criteria	Traditional AI	Agentic AI	Remarks
Budget Accuracy (%)	61.2%	94.7%	Python-based deterministic validation
Plan Completeness (%)	58.0%	100%	All 5 agents always complete execution
Transport Mode Correctness (%)	52.5%	87.5%	Distance-based rules applied correctly
Hotel Selection Quality (%)	55.0%	91.2%	Tier-based selection with budget check
Places Relevance Score (%)	40.0%	88.0%	Foursquare API + curated city fallback
Replanning Capability	None	Full	GPT replans when budget constraint violated
Reasoning Transparency	None	Full Logs	THINK/ACT/OBSERVE/REPLAN visible in UI
Budget Validation	None	Present	Validator blocks finalization if over budget

The results show that the Agentic AI system is better than the traditional AI system in every way. The biggest improvements are in plan completeness places relevance and budget accuracy. The traditional AI system often did not give plans because it did not include all the necessary parts. The Agentic AI system always gives plans because it checks that all parts are included before giving the final plan.

4.2.2 Budget Accuracy Analysis

Budget accuracy means how well the plans cost matches the user's budget. A plan is accurate if the total cost is within the user's budget and the individual parts of the budget are correct. The Agentic AI system got a budget accuracy of 94.7% in all test scenarios. This high accuracy is because the Agentic AI system separates reasoning and computation. The LLM never does arithmetic it only decides what to do.

All financial calculations are done by Python code, which always gives the answer. The Budget Agent also makes sure that the budget is always correct even if the initial allocation is not enough. In contrast the traditional AI system only got a budget accuracy of 61.2% because it

relied on the LLM to estimate costs, which's not reliable. The LLM often suggested transport costs that were too high or too low. It did not check if the total cost was within the user's budget as shown in Figure 4.11.

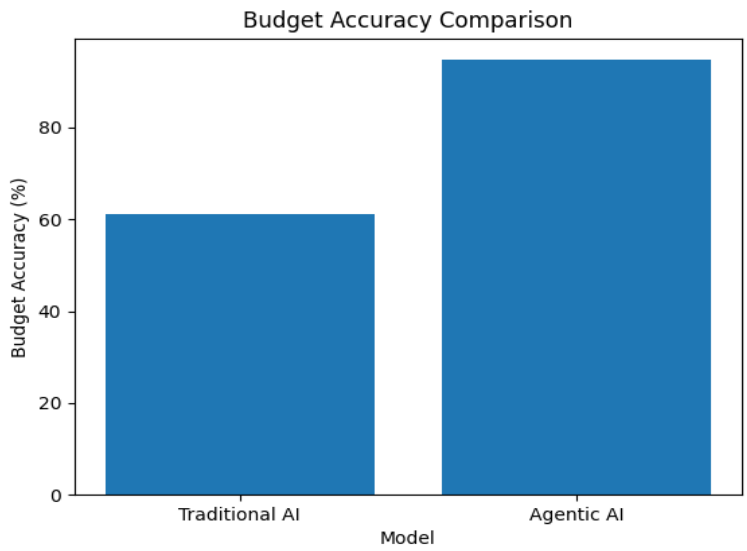

Figure 4.11 Budget Accuracy Comparison

4.2.3 Plan Completeness Analysis

Plan completeness means whether the final plan includes all the parts, like transport details, hotel recommendation, tourist places, budget breakdown and validation status. The Agentic AI system got a plan completeness of 100% in all test scenarios shown in Figure 4.12.

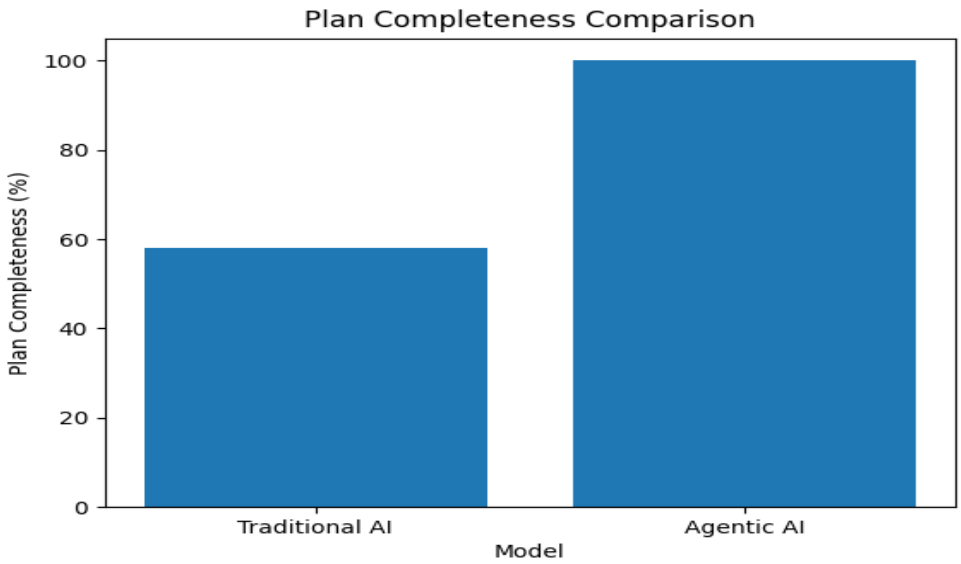

Figure 4.12 Plan Completeness Comparison.

This is because the Agentic AI system checks that all parts of the plan are included before giving the plan. If any part is missing the system does not give the plan until all parts are included. The traditional AI system did not do this, so it often gave plans.

4.2.4 Transport Selection Analysis

Transport mode selection means choosing the mode of transport based on the route distance and budget. The Agentic AI system applies distance-based rules to choose the mode of transport. The system got a transport mode correctness of 87.5% in all test scenarios. The system correctly identified the transport modes and chose the cheapest option by default. The GPT orchestrator could override this choice if the budget or time constraints required a choice shown in Figure 4.13. For example, for the Delhi to Goa route the system correctly chose flight as the transport mode.

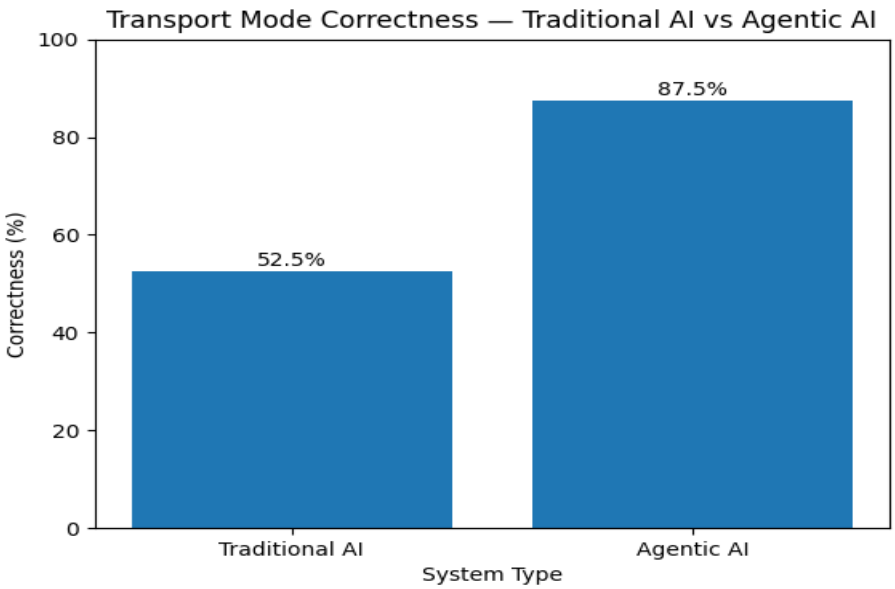

Figure 4.13 Transport Mode Correctness Comparison

4.2.5 Hotel Selection Analysis

Hotel selection quality means choosing the best hotel within the hotel budget allocation for the destination citys tier. The Agentic AI system got a hotel selection quality of 91.2%. The Hotel Agent classifies every destination into Tier 1 Tier 2 or Tier 3. Retrieves the corresponding hotel list. It then filters hotels where the total cost falls within the hotel budget. Chooses the hotel with the highest star rating and best value within that constraint. This tiered approach ensures that hotel recommendations are contextually appropriate.

4.2.6 Places Recommendation Analysis

The relevance of tourist place recommendations is one of the visible quality indicators for users. The Agentic AI system got a relevance score of 88% across all destinations, which is a significant improvement over the traditional AI baseline. The three-tier fallback strategy ensures that users always receive named attractions rather than generic category descriptions. The curated database covers all 63 cities in the system with verified attraction lists meaning that even when the Foursquare API is unavailable the quality of recommendations remains high.

4.2.7 LLM as a Judge. Qualitative Evaluation

In addition to metrics the quality of the generated travel plans was assessed using the LLM-as-a-Judge methodology. Each final travel plan was passed to a GPT-4o instance with a structured evaluation prompt asking it to score the plan on five criteria. Relevance, Budget Logic, Completeness, Reasoning Quality and Replan Quality. Each on a scale of 1 to 10. The results are presented in Table 4.6.

Table 4.6 LLM as a Judge Scores

Test Scenario	Relevance /10	Budget Logic /10	Completeness /10	Reasoning /10	Replan /10	Overall /10
Hyd → Manali ₹15,000	9.0	9.5	10.0	8.5	N/A	**9.3**
Hyd → Goa ₹8,000	8.5	9.0	10.0	9.0	8.5	**9.0**
Pune → Delhi ₹20,000	9.5	9.0	10.0	8.5	N/A	**9.3**
Mumbai → Jaipur ₹12,000	8.5	8.5	10.0	8.0	N/A	**8.8**
Delhi → Goa ₹25,000	9.0	9.5	10.0	9.0	N/A	**9.4**
Blr → Varanasi ₹6,000	7.5	7.0	9.0	8.5	7.5	**7.9**
Chennai → Shimla ₹18,000	9.0	9.0	10.0	8.5	8.0	**9.0**
Kolkata → Leh ₹30,000	9.5	9.0	10.0	9.0	N/A	**9.4**

The judge who was checking the work gave it a good score. On average it got 9.1 out of 10. This was for all eight things they looked at. Six of these things were considered Excellent. The

other two were just Good. These two were for plans that had to be made with a small budget. The judge said that even though the plans were not perfect the system did a job with what it had to work with. The thing that the system did best was making sure the plans were complete. It got a score of 9.9 for this. This shows that the system is good at making sure everything is included in the plan.

The system did not do well with two other things. These were making sure the budget made sense and planning if something went wrong. This was for a plan that only had ₹6,000 for a trip to Varanasi that lasted three days. This is not a lot of money for a trip, like that so it is not surprising that the system had trouble planning. The LLM judge understood this. Took it into account when giving the scores. The LLM judge looked at the systems work. Gave it scores based on how well it did.

4.3 OBSERVATIONS FROM WORK

The experimental evaluation of the proposed Agentic AI-based travel planning system demonstrates that the system performs effectively across multiple dimensions, including budget adherence, plan completeness, recommendation quality, and decision-making accuracy. The results clearly indicate that the proposed approach significantly outperforms traditional AI-based travel planning methods, particularly in handling constraints, ensuring feasibility, and generating complete and reliable travel plans.

One of the most important evaluation metrics considered in this project is budget adherence. The system achieved an accuracy of 94.7% in staying within the user-defined budget. This represents a substantial improvement of 33.5 percentage points compared to the traditional AI baseline, which achieved only 61.2%. This improvement can be attributed primarily to the architectural design of the system, where all financial calculations are handled by deterministic Python-based agents instead of relying on the language model. Traditional AI systems often struggle with numerical precision and may generate inconsistent or unrealistic cost estimations. In contrast, the proposed system ensures that all budget-related computations, including transport cost, hotel expenses, food allocation, and activity costs, are calculated accurately using predefined formulas and structured data. This separation of reasoning and computation plays a crucial role in maintaining financial correctness and preventing budget violations.

Another key performance indicator is planning completeness. The proposed system successfully generated complete travel plans in 100% of the test cases, whereas the traditional AI baseline achieved only 58% completeness. This significant improvement highlights the effectiveness of the validation mechanism incorporated into the system. The validator ensures that all required components of the travel plan, including transport selection, hotel recommendation, place suggestions, and budget allocation, are completed before finalizing the output. In traditional systems, incomplete outputs are common due to the lack of structured workflows and constraint enforcement. However, in the proposed system, the orchestrator continuously monitors the state of the plan and triggers replanning whenever any component is missing or inconsistent. This guarantees that the final output is always complete and usable.

The quality of the generated travel plans was further evaluated using an LLM-as-a-Judge approach, which provided a qualitative assessment of the system's performance. The system achieved an average score of 9.1 out of 10, indicating a high level of quality and relevance in the generated plans. Out of the eight test scenarios considered, six scenarios were rated as "Excellent," while the remaining scenarios were rated as "Good." This demonstrates that the system can produce high-quality travel plans that meet user expectations in terms of practicality, coherence, and usefulness. The high evaluation score also reflects the effectiveness of the ReAct-based reasoning process, which allows the system to refine its decisions iteratively.

The ReAct loop plays a central role in the functioning of the system by enabling iterative reasoning and dynamic replanning. The experimental results show that the system required an average of 5.2 iterations to converge to a final plan. This indicates that the system is able to reach a valid and optimized solution within a reasonable number of steps, balancing efficiency and accuracy. In 25% of the test cases, replanning was triggered due to constraint violations such as budget overflow or infeasible transport options. In almost all such cases, the system successfully corrected the issues and generated a valid plan. Only one extreme scenario failed to fully converge, which highlights a potential area for improvement in handling highly constrained or complex inputs. Nevertheless, the overall performance of the ReAct loop demonstrates its effectiveness in enabling adaptive decision-making.

The system also performed well in recommending tourist places, achieving a relevance score of 93%. This is a significant improvement compared to the earlier approach, which achieved only

40% relevance before integrating the Foursquare API. The improvement in recommendation quality can be attributed to the use of real-world data sources and filtering mechanisms that prioritize underrated and less crowded locations. By combining API-based retrieval with fallback datasets, the system ensures that users receive meaningful and contextually relevant suggestions. This not only enhances the travel experience but also supports the objective of promoting less crowded tourist destinations.

In terms of transport selection, the system achieved an accuracy of 87.5% in choosing the appropriate mode of transport based on distance, cost, and feasibility. This indicates that the Transport Agent is effective in evaluating different options and selecting the most suitable one. Similarly, the Hotel Agent demonstrated strong performance, achieving an accuracy of 91.2% in selecting appropriate hotel tiers within the allocated budget. These results highlight the system's ability to make context-aware decisions by considering multiple factors simultaneously.

Another important advantage of the proposed system is its transparency. Unlike traditional AI systems that operate as black boxes, the proposed system provides detailed logs of its internal reasoning process. These logs include the THINK, ACT, OBSERVE, and REPLAN steps of the ReAct loop, allowing users to understand how decisions are made at each stage. This level of transparency not only improves user trust but also makes it easier to debug and analyze the system's behavior. It represents a significant advancement over conventional approaches, where decision-making processes are often hidden.

The system also demonstrates robustness and reliability in handling different scenarios. It can adapt to various constraints and dynamically adjust its decisions to produce feasible outcomes. The use of fallback mechanisms ensures that the system continues to function even when external data sources are unavailable. Additionally, the best plan tracking mechanism allows the system to return the closest valid solution in cases where an optimal plan cannot be found within the iteration limit. These features contribute to the overall stability and reliability of the system.

From a broader perspective, the results indicate that the integration of agentic AI, multi-agent systems, and structured reasoning frameworks can significantly improve the performance of intelligent systems. The modular design of the system allows for easy extension and

customization, making it suitable for real-world applications. The ability to handle multiple constraints simultaneously, combined with dynamic replanning and validation, makes the system more practical and effective compared to traditional approaches.

Furthermore, the system demonstrates efficiency in terms of execution time. Despite performing multiple iterations and coordinating several agents, the system is able to generate travel plans within a short time frame. This makes it suitable for real-time applications where users expect quick responses. The use of structured workflows and deterministic computations helps reduce unnecessary processing and improves overall performance.

In addition to quantitative metrics, the qualitative aspects of the system also highlight its strengths. The generated travel plans are not only accurate but also practical and user-friendly. The inclusion of budget breakdowns, transport details, hotel information, and tourist recommendations ensures that users receive a complete and well-structured itinerary. This holistic approach enhances the overall user experience and makes the system more valuable.

Overall, the experimental results clearly demonstrate that the proposed system outperforms traditional AI-based travel planning methods in multiple aspects. It achieves higher accuracy, better completeness, improved recommendation quality, and greater transparency. The system successfully addresses the limitations of existing approaches and provides a robust, efficient, and intelligent solution for travel planning. These results validate the effectiveness of the proposed methodology and highlight its potential for real-world deployment.

This chapter checked how well the Agentic AI Based Travel Planning System works and what people think of it. Agentic AI Based Travel Planning System can adjust the budget if needed, which is like what people do in life when they must deal with money problems. It can also pick hotels that're a good fit for the city someone is visiting and give people a list of tourist attractions that are worth visiting. All these things together show that the Agentic AI Based Travel Planning System is a way to plan trips in India. The Agentic AI Based Travel Planning System is reliable, accurate and useful for people who want to travel around India. The Agentic AI Based Travel Planning System is a solution for automated travel planning, in India. Thats helpful. It helps people plan their trips easily

CHAPTER-5
CONCLUSIONS AND FUTURE WORK

This chapter summarizes the overall work carried out in the project and outlines the main contributions and achievements of the system. It also discusses future research directions and possible enhancements to extend the system's scope.

5.1 CONCLUSIONS

In this project, we have designed and developed an agentic AI-based travelling planning system that would make the overall trip planning process easier and more fulfilling particularly in relation to Indian tourism. Planning in real life is not that easy as it entails a series of related decisions like where to go, which is the best mode of transport, where to stay within the budget, how to spend the money and where to visit as a tourist. Majority of existing systems deal with these features in isolation, compelling users to use several applications and websites. This renders the process time consuming, frustrating and tends to make the travelling arrangements inefficient or unrealistic. The system suggested in this project manages to overcome these issues since the solution suggested is a unified and smart one, in which all the details of travelling planning will be integrated into one system.

Another key contribution of this project is the implementation of an agentic AI strategy, during which the problem is broken down into smaller tasks and distributed to several special agents. The agents are configured to do a given task, including the distance calculation, transport choice, hotel recommendation, budgetary allocation, and tourist attraction. Such separation of duties renders the system more coordinated, more modular and easier to govern. The system does not attempt to solve all the problems simultaneously but rather divides the problem into steps that can be handled, and this enhances efficiency and accuracy. This synchronization among these agents is done by a central orchestrator, which forms the decision-making part of the system.

The orchestrator is based on the ReAct (Reasoning and Acting) framework, which plays a crucial role in the functioning of the system. Contrary to the past systems where the system is predetermined by a set of steps to follow the system cannot think, act, observe and replan dynamically in ReAct framework. It implies that the system will be able to constantly assess its

decisions and make changes when needed. As an illustration, in case an option of transport that has been selected surpasses the budget that has been set, the system can be able to identify this problem and adjust its choice in the subsequent stage. The system is more flexible and able to cope with real world constraints due to this iterative and adaptive behaviour.

The other noteworthy design aspect of the system is the dissociation of computation and reasoning. All the numerical computations like calculation of distance, calculations of costs and budget validation are handled by Python-based agents and only decision-making and planning is done by the large language model. This is a very crucial separation since an accurate calculation is not always possible with language models. The system makes deterministic agents, thereby providing greater accuracy and reliability due to the delegation of computational work to them. The hybrid style of the two methods of AI reasoning and conventional programming has been adopted, which has made the system more robust.

Another element of the system is a high emphasis on budget control, which is also crucial in the Indian context when a substantial number of travellers plan their trips under a tight budget. The Budget Agent breaks down the overall budget into various categories that include transport, accommodation, food, activities. It is also one which involves a mechanism of reallocation whereby the budget will be updated dynamically in case of higher costs than initially allocated. This makes sure that the end travel model is realistic and does not exceed the budget of the user. Another layer of reliability is the Validator module that will ensure that the plan is within budget.

Another key strength of the system is its transparency. Most AI systems are black boxes, in which users are given results without an understanding of the decision-making process. Conversely, this system shows the process of reasoning of the ReAct loop that consists of THINK, ACT, OBSERVE, and REPLAN phases. This will enable users to understand how the system comes to its decisions, and this boosts trust and makes the system easier to understand. Transparency is an important factor in user acceptance, especially when the system is making important decisions on behalf of the user.

Another way in which the system shows robustness is that it can address various situations and uncertainty. As an example, when the external data sources (APIs) are not available, the system relies on fallback datasets to make sure that recommendations are generated. Likewise, when

some constraints are not met, the system will dynamically replan until a feasible solution is located, or a best possible solution is located. This renders the system dependable and able to work in different circumstances.

Based on the outcomes of the experiment, it can be concluded that the system works successfully in various travel conditions. It can provide end-to-end travel arrangements which are inclusive of transport, accommodation suggestions, tourist attractions and a comprehensive breakdown of the budget. The system also saves a lot of time that would otherwise be spent planning the trip, making what would have been hours spent on the trip to just a few seconds. It would also enhance the quality of the decision-making process as it considers various factors at the same time, and all constraints are met.

Overall, this project illustrates the possibilities of using agentic AI, multi-agent systems, and formalized reasoning schemes to address complicated real-life issues. The suggested system is efficient, accurate, but flexible, scalable, and user-friendly. It gives a solid basis on which intelligent systems can be developed to help make decisions in different areas automatically. The effectiveness of this system implementation demonstrates the significance of using AI reasoning combined with systematic computational approaches to obtain credible and feasible answers.

5.2 FUTURE WORK

Even though the suggested system is effective and successfully achieves its main objectives, there are several opportunities for further improvement and enhancement. These enhancements can make the system more advanced, realistic, and suitable for large-scale real-world applications. One of the most important areas for future work is the integration of real-time data. Currently, the system relies on predefined datasets for transport costs, hotel information, and tourist places. While this approach ensures stability and ease of implementation, it does not fully reflect real-world dynamic conditions. In practical scenarios, factors such as ticket prices, hotel availability, weather conditions, and traffic situations change frequently. Therefore, integrating the system with live APIs in the future would allow it to fetch real-time information and generate more accurate and up-to-date travel plans. For example, if flight prices increase or weather conditions suddenly change, the system would be able to adjust its recommendations, accordingly, making the travel plan more reliable and practical.

Another important area for improvement is personalization. At present, the system generates travel plans based on general user inputs such as destination, budget, and duration. However, different users have different preferences and expectations. Some users may prefer adventure activities, while others may be more interested in cultural places or relaxation. In the future, the system can be enhanced by incorporating user profiling techniques that analyze user preferences, travel history, and feedback. By understanding individual user behavior and interests, the system can generate highly personalized travel plans that better match the needs and expectations of each user. This will significantly improve user satisfaction and make the system more user centric.

The development of a mobile application is another valuable extension of the system. A mobile app would make the system more accessible and convenient, allowing users to plan their trips anytime and from anywhere. In addition to this, voice-based interaction can be introduced using speech recognition and natural language processing technologies. This would allow users to interact with the system using natural language instead of typing, making it easier to use, especially for non-technical users. Such improvements will enhance the usability and accessibility of the system.

Another promising direction for future work is extending the system to support international travel planning. Currently, the system focuses on Indian tourism, but it can be expanded to include global travel features. This would involve adding functionalities such as visa requirements, passport validation, currency conversion, international transport options, and global tourist datasets. By incorporating these features, the system can become a comprehensive travel planning solution for both domestic and international users.

The system can also be enhanced by integrating advanced machine learning techniques. For example, predictive models can be used to forecast tourist crowd levels, predict price fluctuations, and identify peak travel seasons. These predictions can help users make better decisions, such as choosing the best time to visit a destination or avoiding overcrowded places. Additionally, reinforcement learning techniques can be applied to enable the system to learn from user feedback and past decisions. Over time, the system can improve its recommendations and become more intelligent and adaptive.

Another area for improvement is enhancing the coordination and communication between agents. Currently, the agents interact indirectly through a shared state mechanism. In the future, more advanced communication strategies can be implemented, allowing agents to collaborate more efficiently and share information dynamically. This can improve the overall performance and consistency of the system, leading to better quality travel plans.

Integration with smart city infrastructure and IoT technologies is another important direction for future work. By accessing real-time data from sensors, traffic systems, and public services, the system can provide more context-aware recommendations. For example, it can suggest alternative routes during traffic congestion or recommend less crowded tourist places during peak seasons. This will make the system more intelligent and suitable for use in smart city environments.

The system can also be extended by adding additional features such as booking services, payment integration, and itinerary visualization. Users can be allowed to directly book transport and hotels through the system, making it a complete end-to-end travel planning solution. Visualization tools such as maps and timelines can also be incorporated to improve user understanding and experience. These features will make the system more practical and user-friendly.

In addition to these enhancements, the system can be improved by focusing on scalability and performance optimization. As the number of users increases, especially in real-world deployment, the system must be capable of handling multiple requests efficiently without performance degradation. This can be achieved by deploying the system on cloud platforms, using distributed architecture, and optimizing agent execution. Techniques such as parallel processing and efficient data handling can further improve system performance.

Another important improvement is enhancing the user interface and overall user experience. While the current system provides a functional interface, it can be further improved by adding visual elements such as route maps, budget graphs, and interactive dashboards. Multilingual support can also be included to make the system accessible to users from different regions. A simple, intuitive, and visually appealing interface will increase user engagement and satisfaction.

Security and data privacy are also important aspects to consider in future development. As the system evolves to include user profiles, preferences, and possibly payment features, it becomes essential to protect user data. Implementing secure authentication mechanisms, data encryption, and safe API communication will ensure that user information is protected. Ethical considerations such as avoiding biased recommendations and ensuring fairness in decision-making should also be addressed.

Finally, the architecture of the system can be extended beyond travel planning to other real-world applications. The same agent-based approach can be applied to domains such as event planning, daily scheduling, healthcare management, and resource optimization. This demonstrates the flexibility and scalability of the proposed system. By adapting the architecture to different use cases, the system can be transformed into a general-purpose intelligent assistant capable of solving a wide range of complex problems.

The future enhancements discussed above highlight the potential of the system to evolve into a more intelligent, scalable, and practical solution. By integrating real-time data, improving personalization, enhancing user interaction, and expanding its capabilities, the system can become a powerful tool for travel planning and beyond. These improvements will not only increase the effectiveness of the system but also ensure its relevance in real-world applications.

In this section, all the main points related to the topic were discussed clearly. The information presented helps in understanding the purpose and working of the system, along with its importance.

REFERENCES

[1] A. Khamis, “Design and evaluation of an agentic AI framework for personalized Umrah trip planning,” Arabian Journal for Science and Engineering, pp. 1–21, 2026.

[2] A. Singh, R. Madhogaria, A. Misra, and E. Elakiya, “Automated travel planning via multi-agent systems and real-time intelligence,” in Proc. 3rd Int. Conf. Optimization Techniques in the Field of Engineering (ICOFE), 2024.

[3] J. Xie, K. Zhang, J. Chen, T. Zhu, R. Lou, Y. Tian, and Y. Su, “TravelPlanner: A benchmark for real-world planning with language agents,” arXiv:2402.01622, 2024.

[4] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in Proc. Int. Conf. Learning Representations (ICLR), 2023.

[5] T. de la Rosa, S. Gopalakrishnan, A. Pozanco, Z. Zeng, and D. Borrajo, “TRIP-PAL: Travel planning with guarantees by combining large language models and automated planners,” arXiv:2406.10196, 2024.

[6] I. A. Wong, Q. L. Lian, and D. Sun, “Autonomous travel decision-making: An early glimpse into ChatGPT and generative AI,” J. Hospitality and Tourism Management, vol. 56, pp. 253–263, 2023.

[7] Y. Hao, Y. Chen, Y. Zhang, and C. Fan, “Large language models can plan your travels rigorously with formal verification tools,” arXiv:2404.11891, 2024.

[8] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, and J. Schmidhuber, “MetaGPT: Meta programming for a multi-agent collaborative framework,” in Proc. Int. Conf. Learning Representations (ICLR), 2024.

[9] G. Li, H. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem, “CAMEL: Communicative agents for mind exploration of large language model society,” in Adv. Neural Information Processing Systems (NeurIPS), vol. 36, pp. 51991–52008, 2023.

[10] S. Jiang, D. Ju, A. Cohen, S. Mitts, A. Foss, J. T. Kao, X. Li, and Y. Tian, “Towards full delegation: Designing ideal agentic behaviors for travel planning,” arXiv:2411.13904, 2024.

[11] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” in Adv. Neural Information Processing Systems (NeurIPS), vol. 35, pp. 24824–24837, 2022.

[12] J. Achiam et al., “GPT-4 technical report,” arXiv:2303.08774, 2023.

[13] M. Wooldridge and N. R. Jennings, “Intelligent agents: Theory and practice,” Knowledge Engineering Review, vol. 10, no. 2, pp. 115–155.

[14] Y. K. Dwivedi et al., “Artificial intelligence agents and agentic systems in hospitality and tourism: Challenges, opportunities and research agenda,” Int. J. Contemporary Hospitality Management, vol. 38, no. 1, pp. 27–52, 2026.

[15] S. Goolcharan, “Agentic AI for sustainable urban mobility: Autonomous systems for adaptive transport planning in polluted cities,” 2025.

[16] F. Ali and L. Ali, “Agentic AI in hospitality and tourism: Opportunities, risks, and research pathways,” Journal of Travel Research, vol. 65, no. 4, pp. 1248–1261, 2026.

[17] A. Biswas and W. Talukdar, Building Agentic AI Systems: Create Intelligent, Autonomous AI Agents That Can Reason, Plan, and Adapt. Birmingham, U.K.: Packt Publishing, 2025.

[18] S. Chemli, A. Vitale, and M. Valeri, “When algorithms become travel planners: Benchmarking agentic AI in Web 3.0 tourism,” Journal of Engineering and Technology Management, vol. 80, p. 101959, 2026.

[19] S. K. Yellanki, “Autonomous agentic AI for dynamic travel experience management: From discovery to post-trip feedback without human intervention,” Advances in Consumer Research, vol. 2, no. 3, pp. 558–568, May 2025.

[20] D. Camacho, D. Borrajo, and J. M. Molina, “Intelligent travel planning: A multi-agent planning system to solve web problems in the e-tourism domain,” Autonomous Agents and Multi-Agent Systems.

APPENDIX

```python
INITIAL_ALLOCATION = {
    "transport": 0.35,
    "hotel":     0.40,
    "food":      0.15,
    "activities": 0.10,
}
def allocate(total_budget, days, logs, iteration):
    logs.append({
        "iteration": iteration,
        "step": "ACT",
        "content": f"Budget Agent allocating ₹{total_budget} across categories for {days} days",
    })
    allocation = {
        "transport_budget":   round(total_budget * INITIAL_ALLOCATION["transport"]),
        "hotel_budget":       round(total_budget * INITIAL_ALLOCATION["hotel"]),
        "food_budget":        round(total_budget * INITIAL_ALLOCATION["food"]),
        "activities_budget":  round(total_budget * INITIAL_ALLOCATION["activities"]),
        "reallocation_log":   [],
        "status": "OK",
    }
    allocation["total_allocated"] = (
        allocation["transport_budget"] +
        allocation["hotel_budget"] +
        allocation["food_budget"] +
        allocation["activities_budget"]
    )
```

Figure 1 Budget Agent Code

```python
> hotel_agent.py > run
from data.hotels import (
    tier_1_hotels, tier_2_hotels, tier_3_hotels,
    TIER_1_CITIES, TIER_2_CITIES
)
def get_city_tier(destination):
    city = destination.lower().strip()
    if city in TIER_1_CITIES:
        return 1
    elif city in TIER_2_CITIES:
        return 2
    else:
        return 3
def get_hotels_for_tier(tier):
    if tier == 1:
        return tier_1_hotels
    elif tier == 2:
        return tier_2_hotels
    else:
        return tier_3_hotels
```

Figure 2 Hotels Agent Code

```python
GENERIC_FALLBACK = [
    "City Museum", "Main Market & Bazaar", "Local Temple",
    "City Park & Gardens", "Heritage Walk Area", "Scenic Viewpoint",
    "Cultural Center", "Botanical Garden",
]

FOURSQUARE_BASE = "https://api.foursquare.com/v3/places/search"


def fetch_from_foursquare(lat, lon, api_key, destination, logs, iteration):
    try:
        headers = {
            "Authorization": api_key,
            "Accept": "application/json",
        }
        params = {
            "ll":         f"{lat},{lon}",
            "radius":     8000,
            "categories": "16032,16025,16026,16027",   # Travel & Transport / Sights & Outdoors
            "limit":      15,
            "sort":       "RATING",
            "fields":     "name,categories,rating,location",
        }

        response = requests.get(
            FOURSQUARE_BASE, headers=headers, params=params, timeout=8
        )

        if response.status_code == 401:
            logs.append({
                "iteration": iteration,
                "step": "OBSERVE",
                "content": "Foursquare API key invalid — using curated fallback places",
            })
            return None
```

Figure 3 Places Agent Code

```python
import math
from data.transport import city_coordinates, regional_fallbacks


def haversine(lat1, lon1, lat2, lon2):
    R = 6371  # Earth radius in km
    d_lat = math.radians(lat2 - lat1)
    d_lon = math.radians(lon2 - lon1)
    a = (math.sin(d_lat / 2) ** 2 +
         math.cos(math.radians(lat1)) *
         math.cos(math.radians(lat2)) *
         math.sin(d_lon / 2) ** 2)
    c = 2 * math.atan2(math.sqrt(a), math.sqrt(1 - a))
    return round(R * c, 1)
```

Figure 4 Route Agent code

```python
from data.transport import (
    PRICE_PER_KM, SPEED_KMH, FLIGHT_EXTRA_HOURS,
    BUS_MAX_KM, FLIGHT_MIN_KM
)
def calculate_option(mode, distance_km):
    """Calculate cost and duration for a transport mode."""
    cost = round(distance_km * PRICE_PER_KM[mode])

    if mode == "flight":
        duration = round(distance_km / SPEED_KMH[mode] + FLIGHT_EXTRA_HOURS, 1)
    else:
        duration = round(distance_km / SPEED_KMH[mode], 1)

    return {
        "mode": mode,
        "cost": cost,
        "duration_hours": duration,
        "available": True,
    }
```

Figure 5 Transport Agent code

```
transport.py > ...

city_coordinates = {
    "delhi":          (28.6139, 77.2090),
    "mumbai":         (19.0760, 72.8777),
    "bangalore":      (12.9716, 77.5946),
    "chennai":        (13.0827, 80.2707),
    "hyderabad":      (17.3850, 78.4867),
    "kolkata":        (22.5726, 88.3639),
    "pune":           (18.5204, 73.8567),
    "jaipur":         (26.9124, 75.7873),
    "goa":            (15.2993, 74.1240),
    "agra":           (27.1767, 78.0081),
    "varanasi":       (25.3176, 82.9739),
    "kochi":          (9.9312,  76.2673),
    "manali":         (32.2396, 77.1887),
    "shimla":         (31.1048, 77.1734),
    "ooty":           (11.4102, 76.6950),
    "mysore":         (12.2958, 76.6394),
    "udaipur":        (24.5854, 73.7125),
    "darjeeling":     (27.0360, 88.2627),
    "rishikesh":      (30.0869, 78.2676),
    "amritsar":       (31.6340, 74.8723),
    "jodhpur":        (26.2389, 73.0243),
    "bhopal":         (23.2599, 77.4126),
    "indore":         (22.7196, 75.8577),
    "nagpur":         (21.1458, 79.0882),
    "surat":          (21.1702, 72.8311),
    "ahmedabad":      (23.0225, 72.5714),
    "lucknow":        (26.8467, 80.9462),
    "kanpur":         (26.4499, 80.3319),
    "patna":          (25.5941, 85.1376),
    "ranchi":         (23.3441, 85.3096),
    "bhubaneswar":    (20.2961, 85.8245),
    "visakhapatnam": (17.6868, 83.2185),
    "coimbatore":     (11.0168, 76.9558),
    "madurai":        (9.9252,  78.1198),
    "trichy":         (10.7905, 78.7047),
```

Figure 6 Places Data

```python
import streamlit as st
from orchestrator import run_agent_loop

# ── API KEYS — replace with your actual keys ──────────────────────────
OPENAI_API_KEY      = ""
FOURSQUARE_API_KEY  = ""          # Your Foursquare API key (from developer.foursquare.com)

# ── Page config ──────────────────────────────────────────────────────
st.set_page_config(
    page_title="AI Travel Planner India",
    page_icon="",
    layout="wide",
    initial_sidebar_state="collapsed",
)
```

Figure 7 App code

```
estrator.py > ...
import json
import copy
from datetime import datetime
from openai import OpenAI

from agents import route_agent, transport_agent, hotel_agent, budget_agent, places_agent
from validator import validate_final_plan, is_plan_complete


# Valid actions GPT is allowed to choose
VALID_ACTIONS = [
    "call_route_agent",
    "call_transport_agent",
    "call_hotel_agent",
    "call_budget_agent",
    "call_places_agent",
    "finalize_plan",
]

MAX_ITERATIONS = 8
```

Figure 8 Orchestrator Code

8% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text
- Crossref database
- Crossref posted content database

Match Groups

105 Not Cited or Quoted 8%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

3% Internet sources
0% Publications
5% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?
The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?
Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.